



Universidad Técnica Estatal de Quevedo

Facultad de ciencias de la computación

Ingeniería en software

Asignatura:

Ingeniería de requerimientos

Tema:

GA – Exposición Segundo Corte: Herramientas CASE

Estudiante:

Contreras Chavez Kevin German

CL:1251167720

Correo: kcontrerasc3@uteq.edu.ec

Docente:

PhD. Guerrero Ulloa Gleiston Ciceron

Fecha:

18/07/2026

REGULAR - 2026-2027 PPA

Link GitHub:

<https://github.com/avaldiviezoy-hub/PFC-UTeqMarket-MDD-EquipoD>

Resumen

UTeqMarket es una plataforma web orientada a la compra y venta de productos y servicios entre los miembros de la Universidad Técnica Estatal de Quevedo (UTEQ), con el objetivo de ofrecer un entorno seguro mediante la validación de usuarios con correo institucional. Para demostrar la aplicación del enfoque Model-Driven Development (MDD), se utilizó la herramienta PowerDesigner, la cual permitió modelar un subsistema representativo del proyecto mediante diagramas UML y generar automáticamente código fuente a partir del modelo diseñado. El módulo seleccionado para la demostración corresponde a la gestión de publicaciones, incluyendo las entidades principales y sus relaciones dentro del sistema. Como lenguaje de programación objetivo se empleó C#, generado automáticamente por la herramienta a partir del modelo orientado a objetos. Como resultado, se obtuvo un modelo UML consistente con los requisitos funcionales del proyecto, código fuente estructurado y trazable respecto al modelo, además de evidenciar el proceso de generación automática y su posterior verificación. Esto permitió comprobar la utilidad del enfoque MDD para mejorar la productividad, mantener la consistencia entre el diseño y la implementación, y facilitar la evolución del sistema mediante cambios controlados en el modelo.

Marco teórico

Model-Driven Engineering

El Model-Driven Engineering (MDE) es un paradigma de desarrollo de software que considera a los modelos como los principales artefactos durante el ciclo de vida del sistema. En lugar de utilizar los modelos únicamente como documentación, MDE los convierte en la fuente principal para la construcción, transformación y mantenimiento del software mediante herramientas automatizadas. Este enfoque busca elevar el nivel de abstracción del desarrollo, permitiendo que los desarrolladores se concentren en la lógica del negocio antes que en los detalles específicos de implementación [1].

El objetivo principal de MDE es mejorar la productividad, reducir errores y facilitar el mantenimiento de los sistemas mediante la automatización de tareas repetitivas, como la generación de código fuente o documentación. Además, promueve la reutilización de modelos y favorece la trazabilidad entre los requisitos, el diseño y la implementación del software[2].

Model-Driven Architecture

La Model-Driven Architecture (MDA) es una iniciativa propuesta por el Object Management Group (OMG) que define una arquitectura basada en modelos para desarrollar aplicaciones independientes de la plataforma tecnológica. Su propósito es separar la lógica del negocio de los detalles específicos de implementación, permitiendo que un mismo modelo pueda adaptarse a diferentes tecnologías mediante transformaciones automáticas [3].

MDA organiza el desarrollo en tres niveles de abstracción:

Computation Independent Model (CIM): representa el contexto del negocio y los requisitos del sistema sin considerar aspectos tecnológicos [4].

Platform Independent Model (PIM): describe la estructura y el comportamiento del sistema sin depender de una plataforma específica [4].

Platform Specific Model (PSM): incorpora las características propias de una tecnología o lenguaje de programación determinado, a partir del cual se genera el código fuente [4].

Esta separación favorece la reutilización de modelos, la portabilidad del software y la reducción del esfuerzo necesario para migrar una aplicación entre diferentes plataformas.

Model-Driven Development (MDD)

El Model-Driven Development (MDD) corresponde a la aplicación práctica de los principios de MDE durante el desarrollo de software. En este enfoque, los modelos UML sirven como base para generar automáticamente parte o la totalidad del código fuente mediante herramientas especializadas [5].

El ciclo de desarrollo comienza con la elaboración del modelo conceptual, continúa con su transformación hacia modelos específicos de plataforma y finaliza con la generación automática del código correspondiente. Durante este proceso también pueden realizarse modificaciones sobre el modelo para regenerar el código cuando sea necesario [6].

Entre sus principales beneficios se encuentran la disminución del tiempo de desarrollo, la reducción de errores manuales, la mejora de la consistencia entre diseño e implementación y el incremento de la productividad. Sin embargo, también presenta algunas limitaciones, como la dependencia de herramientas especializadas, la necesidad de modelos correctamente definidos y las restricciones impuestas por los generadores automáticos de código [7].

Transformaciones de modelos

Transformaciones Modelo a Modelo (M2M)

Las transformaciones Modelo a Modelo (Model-to-Model, M2M) consisten en convertir un modelo origen en otro modelo con un mayor nivel de detalle o adaptado a una plataforma específica, manteniendo la información esencial del sistema [8]. Un ejemplo es la transformación de un modelo independiente de la plataforma (PIM) hacia un modelo específico de plataforma (PSM), incorporando elementos propios del entorno tecnológico donde será implementado el software [9].

Transformaciones Modelo a Texto (M2T)

Las transformaciones Modelo a Texto (Model-to-Text, M2T) permiten generar automáticamente artefactos textuales a partir de un modelo, como código fuente, scripts de bases de datos,

documentación o archivos de configuración [10]. En herramientas como PowerDesigner, estas transformaciones convierten los diagramas UML en código fuente en lenguajes como C#, Java o C++, reduciendo el trabajo manual y garantizando la correspondencia entre el modelo y la implementación [1].

Metamodelos

Un metamodelo es un modelo que define la estructura, las reglas y la sintaxis de otros modelos. En el contexto de UML, el estándar Meta Object Facility (MOF) proporciona la base sobre la cual se define el lenguaje UML, especificando los elementos que pueden utilizarse para representar sistemas de software [11].

Gracias a MOF, UML mantiene una estructura estandarizada que facilita el intercambio de modelos entre diferentes herramientas CASE y permite que los generadores automáticos interpreten correctamente las clases, atributos, asociaciones y demás elementos del modelo [2].

Perfiles UML

Los perfiles UML constituyen un mecanismo de extensión del lenguaje UML que permite adaptar los modelos a dominios o tecnologías específicas sin modificar el estándar original [1]. Entre sus principales elementos se encuentran los estereotipos, que añaden nuevos significados a los elementos UML, y los Tagged Values, que permiten incorporar información adicional mediante atributos personalizados [2].

Estos mecanismos facilitan que las herramientas de generación automática interpreten correctamente el propósito de cada elemento y produzcan código acorde con la plataforma objetivo [3].

Plantillas de generación

Las plantillas de generación (templates) son reglas utilizadas por las herramientas MDD para transformar los elementos del modelo en código fuente u otros artefactos. Cada plantilla establece cómo debe convertirse una clase, un atributo, una operación o una relación UML al lenguaje de programación seleccionado [12].

Los generadores de código utilizan estas plantillas para automatizar la construcción del software, reduciendo errores humanos y asegurando la consistencia entre el modelo y la implementación [13].

Ingeniería Forward, Reverse y Round-Trip

Tipo de ingeniería	Descripción
Forward Engineering	Genera automáticamente código fuente a partir de modelos UML [13].
Reverse Engineering	Obtiene modelos UML a partir del código fuente existente [14].
Round-Trip Engineering	Mantiene sincronizados el modelo y el código, permitiendo regenerar el software cuando se realizan cambios en cualquiera de los dos [15].

Justificación de la herramienta

La selección de una herramienta CASE es un aspecto importante dentro del enfoque Model-Driven Development (MDD), ya que de ella depende la capacidad para modelar sistemas, generar código fuente y mantener la consistencia entre el diseño y la implementación. Para el presente trabajo se analizaron distintas herramientas ampliamente utilizadas en ingeniería de software, considerando características como la generación automática de código, el soporte para ingeniería inversa, los lenguajes de programación compatibles y el costo de licenciamiento.

Entre las alternativas evaluadas se encuentran PowerDesigner, Enterprise Architect y Visual Paradigm, todas reconocidas por ofrecer soporte para UML y diversas funcionalidades de modelado. Sin embargo, presentan diferencias en cuanto a sus capacidades de automatización, compatibilidad con lenguajes de programación y enfoque de uso.

PowerDesigner fue seleccionado debido a que proporciona un entorno integrado para el modelado UML, la generación automática de código fuente (Forward Engineering), la ingeniería inversa

(Reverse Engineering) y el soporte para múltiples lenguajes de programación y sistemas gestores de bases de datos. Estas características facilitan la aplicación de los principios de MDD, reducen el esfuerzo manual durante el desarrollo y permiten mantener una adecuada trazabilidad entre el modelo y la implementación.

Además, PowerDesigner ofrece mecanismos de personalización mediante plantillas de generación y mantiene una estructura organizada para la administración de modelos complejos, lo que resulta adecuado para demostrar el proceso de transformación de modelos hacia código fuente requerido en esta práctica.

Herramientas analizadas

Herramienta	Genera código	Ingeniería inversa	Precio	Lenguajes compatibles
PowerDesigner	Sí	Sí	Licencia comercial	C#, Java, C++, Visual Basic, PowerBuilder, entre otros.
Enterprise Architect	Sí	Sí	Licencia comercial (costo medio)	C#, Java, C++, PHP, Python, JavaScript, VB.NET, entre otros.
Visual Paradigm	Sí	Sí	Ediciones Community y comerciales	C#, Java, C++, PHP, Python, JavaScript, Kotlin, entre otros.

¿Por qué PowerDesigner?

PowerDesigner fue seleccionado como herramienta CASE para el desarrollo de la presente práctica debido a las funcionalidades que ofrece para aplicar los principios del Model-Driven Development (MDD). La herramienta proporciona un entorno integrado que permite diseñar modelos UML, generar código fuente, realizar ingeniería inversa y administrar modelos de bases de datos dentro de un mismo proyecto.

Una de sus principales ventajas es su compatibilidad con el estándar UML, lo que permite modelar la estructura y el comportamiento de un sistema mediante diagramas estandarizados ampliamente utilizados en la ingeniería de software. Esto facilita la comunicación entre los miembros del equipo y garantiza que los modelos puedan ser interpretados por diferentes herramientas compatibles con UML.

PowerDesigner también incorpora funciones de generación automática de código (Forward Engineering), permitiendo transformar modelos UML en código fuente de distintos lenguajes de programación. Esta característica reduce el trabajo manual, mejora la productividad y disminuye la posibilidad de errores durante la implementación del sistema.

Otra funcionalidad importante es la ingeniería inversa (Reverse Engineering), mediante la cual es posible obtener modelos UML a partir de código fuente o estructuras de bases de datos ya existentes. Esto facilita la comprensión, documentación y mantenimiento de aplicaciones desarrolladas previamente.

Asimismo, la herramienta ofrece una integración eficiente con bases de datos, permitiendo diseñar modelos conceptuales, lógicos y físicos, así como generar automáticamente scripts SQL para diferentes sistemas gestores de bases de datos. Esta funcionalidad contribuye a mantener la consistencia entre el modelo de datos y la implementación.

PowerDesigner soporta diversos lenguajes de programación, entre ellos C#, Java y SQL, además de otros como C++, Visual Basic y PowerBuilder. Esta compatibilidad proporciona flexibilidad para desarrollar aplicaciones en distintos entornos tecnológicos y facilita la adaptación del proyecto a diferentes plataformas de desarrollo.

En conjunto, estas características convierten a PowerDesigner en una herramienta adecuada para implementar el enfoque de desarrollo dirigido por modelos, favoreciendo la automatización, la trazabilidad y la consistencia entre el análisis, el diseño y la implementación del software.

Relación con UTeqMarket

La utilización de PowerDesigner aporta múltiples beneficios al desarrollo del proyecto UTeqMarket, ya que permite representar de manera estructurada los componentes principales del sistema antes de iniciar la implementación. Mediante el uso de diagramas UML es posible modelar entidades como usuarios, publicaciones, categorías, calificaciones y demás elementos que conforman la plataforma, facilitando la comprensión de la arquitectura del software.

La capacidad de generar código automáticamente resulta especialmente útil para el proyecto, ya que permite convertir el modelo de clases en código base de C#, reduciendo el tiempo de desarrollo y asegurando una mayor correspondencia entre el diseño y la implementación. De esta forma, las modificaciones realizadas sobre el modelo pueden reflejarse con mayor facilidad en el código generado, manteniendo la consistencia del sistema.

Otra ventaja importante es el soporte para el modelado de bases de datos, lo cual permite diseñar la estructura de almacenamiento de UTeqMarket y generar los scripts SQL necesarios para su implementación. Esto contribuye a mantener la integridad de los datos y simplifica la administración del esquema de la base de datos durante el desarrollo.

Además, las funciones de ingeniería inversa permiten reconstruir modelos UML a partir del código o de la base de datos cuando sea necesario actualizar la documentación del proyecto o analizar cambios realizados durante el desarrollo. Esta característica facilita el mantenimiento y la evolución del sistema a lo largo del tiempo.

En consecuencia, PowerDesigner constituye una herramienta que respalda el enfoque Model-Driven Development (MDD) aplicado en UTeqMarket, al integrar el modelado UML, la generación automática de código y el diseño de bases de datos en un único entorno de trabajo. Esto favorece una mejor organización del proyecto, incrementa la productividad del desarrollo y mejora la trazabilidad entre los requisitos, el modelo y la implementación.

Descripción de proyecto

Descripción general

UTeqMarket es una plataforma web desarrollada para facilitar la compra y venta de productos y servicios entre los miembros de la Universidad Técnica Estatal de Quevedo (UTEQ). Su propósito es ofrecer un espacio seguro y organizado donde estudiantes, docentes y personal administrativo puedan publicar anuncios, buscar productos y establecer contacto con otros usuarios pertenecientes a la comunidad universitaria.

Una de las principales características del sistema es la restricción de acceso mediante el uso de correos electrónicos institucionales con dominio @uteq.edu.ec, lo que permite verificar que los usuarios formen parte de la universidad y reduce el riesgo de fraudes o cuentas falsas. Además, la plataforma incorpora funcionalidades para la gestión de publicaciones, búsqueda de productos, filtrado por categorías, visualización de información detallada, comunicación entre comprador y vendedor y un sistema de calificaciones para fortalecer la confianza entre los usuarios.

Desde el punto de vista del desarrollo de software, UTeqMarket constituye un caso de estudio adecuado para aplicar el enfoque Model-Driven Development (MDD), ya que su estructura orientada a objetos permite representar el sistema mediante modelos UML y generar automáticamente parte del código fuente utilizando herramientas CASE como **PowerDesigner**.

Problema

Actualmente, muchos estudiantes, docentes y personal administrativo de la Universidad Técnica Estatal de Quevedo utilizan redes sociales, grupos de mensajería instantánea y plataformas de comercio general para comprar, vender o intercambiar productos y servicios relacionados con la vida universitaria. Sin embargo, estos medios presentan diversas limitaciones, como la falta de verificación de identidad de los usuarios, la presencia de personas externas a la institución, la desorganización de las publicaciones y la ausencia de mecanismos de reputación que permitan generar confianza entre compradores y vendedores.

Estas condiciones incrementan el riesgo de fraudes, dificultan la localización de productos y reducen la seguridad de las transacciones realizadas dentro de la comunidad universitaria. Además,

al no existir una plataforma especializada para la UTEQ, la información suele encontrarse dispersa en múltiples grupos o páginas, lo que disminuye la eficiencia en la comunicación entre los usuarios.

Ante esta situación, surge la necesidad de desarrollar una plataforma que centralice las publicaciones, valide la identidad institucional de los usuarios y proporcione un entorno más seguro, organizado y confiable para el intercambio de bienes y servicios dentro de la universidad.

Objetivo general

Desarrollar el modelado del proyecto UTEqMarket mediante la herramienta PowerDesigner, aplicando los principios del Model-Driven Development (MDD) para representar la estructura del sistema mediante diagramas UML y generar automáticamente código fuente, demostrando la trazabilidad entre el modelo y la implementación como apoyo al desarrollo de una plataforma web destinada al intercambio seguro de productos y servicios entre los miembros de la Universidad Técnica Estatal de Quevedo.

Alcance

El presente trabajo comprende el modelado de un subsistema representativo de UTEqMarket mediante PowerDesigner, aplicando el enfoque Model-Driven Development. Se elaboró un Modelo Orientado a Objetos con las clases principales del dominio, sus atributos, operaciones, asociaciones, cardinalidades y enumeraciones.

A partir del modelo se realizó la generación automática de código fuente en C#, el cual fue incorporado y verificado en Microsoft Visual Studio. También se desarrolló el Modelo Físico de Datos y se generó el script SQL correspondiente para Microsoft SQL Server.

Como parte de la demostración, se comprobó la compilación del código generado, se creó un proyecto de consola para verificar el uso de las clases y se ejecutó un proceso de Round-Trip Engineering mediante la modificación y regeneración de la clase Calificacion.

El trabajo no contempla la implementación completa de la plataforma web, la interfaz de usuario, el acceso definitivo a datos ni la lógica funcional de todos los métodos generados.

Requisitos funcionales relacionados

Para delimitar el componente de persistencia del modelo, se consideraron los siguientes requisitos funcionales relacionados con el almacenamiento y la gestión de información:

Código	Requisito funcional
RF02	Validar el correo institucional del usuario para permitir el acceso al sistema.
RF03	Registrar una nueva publicación de producto o servicio.
RF04	Editar la información de una publicación existente.
RF05	Eliminar una publicación.
RF06	Buscar publicaciones disponibles.
RF07	Filtrar publicaciones por categoría u otros criterios.
RF08	Visualizar la información detallada de una publicación.
RF10	Registrar la calificación de un usuario después de una transacción.
RF11	Marcar una publicación como vendida o finalizada.

Los requisitos funcionales relacionados con la autenticación, la comunicación mediante WhatsApp, la interfaz de usuario y otros procesos externos no forman parte del alcance del modelo entidad-relación, ya que corresponden a funcionalidades implementadas en la lógica de negocio o mediante servicios externos y no afectan directamente la estructura de la base de datos.

Actores

Los actores representan los tipos de usuarios que interactúan con el sistema UTEqMarket. Cada uno posee permisos específicos sobre la información almacenada en la base de datos y participa en diferentes procesos relacionados con la compra y venta de productos y servicios dentro de la comunidad universitaria.

Actor	Descripción	Participación en el sistema
Estudiante	Usuario perteneciente a la UTEQ con correo institucional válido.	Puede publicar productos o servicios, buscar publicaciones, contactar vendedores, realizar compras y calificar usuarios.
Docente	Miembro del personal académico de la UTEQ registrado mediante correo institucional.	Puede publicar, comprar, vender y calificar usuarios dentro de la plataforma.
Personal administrativo	Empleado administrativo de la UTEQ con acceso mediante correo institucional.	Puede publicar productos o servicios, realizar compras, consultar publicaciones y calificar usuarios.
Administrador	Responsable de la administración del sistema.	Gestiona usuarios, supervisa publicaciones, administra categorías y mantiene el correcto funcionamiento de la plataforma.

4. Modelo UML

El modelo UML de UTEQMarket describe la estructura estática del sistema mediante clases, atributos, operaciones, enumeraciones y relaciones. El modelo fue elaborado en PowerDesigner a partir de los requisitos funcionales, reglas de negocio y entidades identificadas para la plataforma. La presente sección documenta el Diagrama de Clases y su correspondencia con el modelo físico implementado en Microsoft SQL Server.

4.1 Diagrama de clases

El Diagrama de Clases representa los principales elementos del dominio de UTEQMarket y la forma en que colaboran entre sí. El modelo está compuesto por las clases Usuario, Publicacion, Categoria, ImagenPublicacion, Favorito, Contacto, Calificacion y Reporte. Además, utiliza las enumeraciones RolUsuario, EstadoPublicacion, EstadoReporte y MotivoReporte para restringir los valores permitidos en determinados atributos.

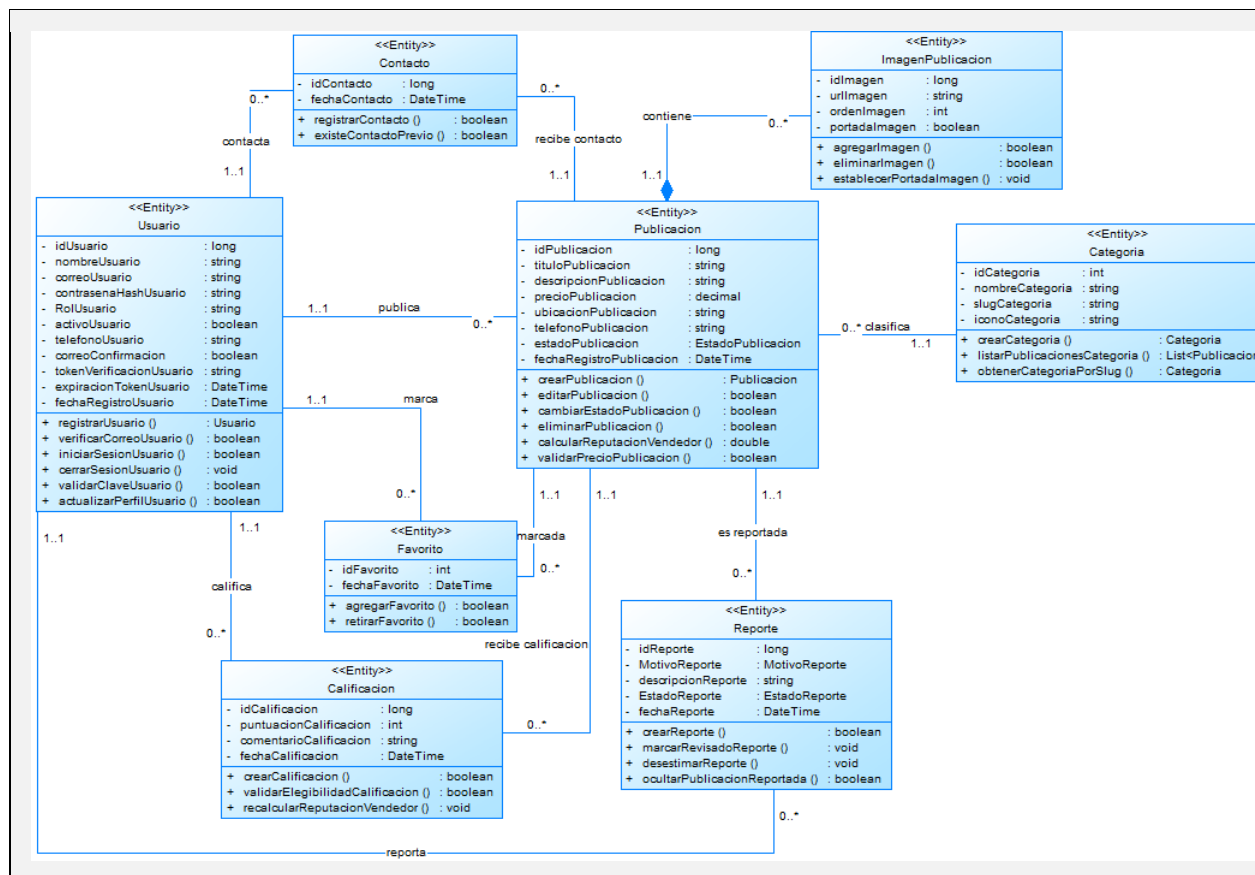


Figura 4.1. Diagrama de Clases UML final de UTEQMarket.

Fuente: Elaboración propia mediante PowerDesigner

Explicación del diagrama

La clase Usuario representa a los miembros de la comunidad universitaria que utilizan la plataforma. Mantiene la información de la cuenta, correo institucional, contraseña, teléfono, rol, estado de activación y datos de verificación. Sus operaciones permiten registrar usuarios, verificar el correo, iniciar y cerrar sesión, validar la contraseña y actualizar el perfil.

La clase Publicacion representa los productos o servicios ofrecidos en la plataforma. Contiene el título, descripción, precio, ubicación, teléfono de contacto, estado y fecha de registro. Sus operaciones permiten crear, modificar, eliminar y cambiar el estado de una publicación, además de validar el precio y calcular la reputación del vendedor.

La clase Categoria organiza las publicaciones según el tipo de producto o servicio. ImagenPublicacion almacena las imágenes asociadas a una publicación, su orden de presentación y la indicación de portada. Favorito, Contacto, Calificacion y Reporte registran las diferentes interacciones realizadas por los usuarios sobre las publicaciones.

Relaciones, herencia y tipos de vínculo

Las relaciones Usuario–Publicacion y Categoria–Publicacion son asociaciones de uno a muchos. Las clases Favorito, Contacto, Calificacion y Reporte funcionan como clases asociativas que resuelven conceptualmente las relaciones muchos a muchos entre Usuario y Publicacion mediante dos asociaciones de uno a muchos.

La relación entre Publicacion e ImagenPublicacion se modela mediante composición, porque la imagen depende del ciclo de vida de la publicación. El modelo no utiliza agregación ni generalización. Los roles comprador, vendedor y administrador se representan mediante la enumeración RolUsuario, evitando crear subclases que dupliquen atributos y operaciones.

4.2 Descripción de clases

Las clases del Diagrama de Clases distribuyen las responsabilidades del sistema de forma coherente. La Tabla 4.1 resume la responsabilidad principal de cada clase.

Tabla 4.1. Descripción de las clases de UTEQMarket.

Clase	Responsabilidad
Usuario	Representar a los miembros de la comunidad UTEQ. Administra la cuenta, autenticación, verificación del correo, rol, estado y datos del perfil.
Publicacion	Representar los productos o servicios ofrecidos. Gestiona título, descripción, precio, ubicación, contacto, estado y fecha de registro.
Categoria	Clasificar y organizar las publicaciones según el tipo de producto o servicio ofrecido.
ImagenPublicacion	Gestionar las imágenes asociadas a una publicación, incluyendo su URL, orden y condición de portada.
Favorito	Registrar la relación entre un usuario y una publicación guardada como favorita.
Contacto	Registrar las solicitudes de contacto realizadas por un usuario sobre una publicación.
Calificacion	Registrar puntuaciones y comentarios emitidos por los usuarios, aportando información para la reputación del vendedor.
Reporte	Registrar y administrar incidencias notificadas sobre una publicación para su revisión administrativa.

Fuente: Elaboración propia.

Tabla 4.2. Enumeraciones utilizadas en el modelo.

Enumeración	Responsabilidad
RolUsuario	Define los roles válidos: COMPRADOR, VENDEDOR y ADMINISTRADOR.
EstadoPublicacion	Restringe el estado de una publicación a ACTIVA, PAUSADA o VENDIDA.
EstadoReporte	Define los estados del reporte: PENDIENTE, REVISADO o DESESTIMADO.
MotivoReporte	Define los motivos permitidos para reportar una publicación.

Fuente: Elaboración propia.

4.3 Atributos

Los atributos representan el estado interno de las clases. En el modelo UML se definieron con visibilidad privada, representada mediante el símbolo -, para aplicar el principio de encapsulamiento. Los campos foráneos creados posteriormente en el modelo físico no se incluyen como atributos del Diagrama de Clases cuando la relación ya se encuentra representada mediante una asociación.

Nota técnica. Los identificadores físicos como idUsuarioPublicacion, idCategoriaPublicacion o idPublicacionFavorito pertenecen al PDM y a SQL Server. En el OOM, las relaciones se muestran mediante asociaciones y cardinalidades.

Tabla 4.3. Atributos de las clases de UTEQMarket.

Clase	Atributo	Tipo	Visibilidad	Descripción
Usuario	idUsuario	long	private (-)	Identificador único del usuario.
Usuario	nombreUsuario	string	private (-)	Nombre completo del usuario.
Usuario	correoUsuario	string	private (-)	Correo institucional utilizado para registro y autenticación.
Usuario	contrasenaHashUsuario	string	private (-)	Contraseña almacenada mediante hash.
Usuario	rolUsuario	RolUsuario	private (-)	Rol asignado dentro de la plataforma.
Usuario	activoUsuario	boolean	private (-)	Indica si la cuenta está activa.
Usuario	telefonoUsuario	string	private (-)	Número de teléfono o WhatsApp.
Usuario	correoConfirmacion	boolean	private (-)	Indica si el correo fue confirmado.
Usuario	tokenVerificacionUsuario	string	private (-)	Token temporal para verificar la cuenta.
Usuario	expiracionTokenUsuario	DateTime	private (-)	Fecha y hora de expiración del token.
Usuario	fechaRegistroUsuario	DateTime	private (-)	Fecha y hora de creación de la cuenta.
Publicacion	idPublicacion	long	private (-)	Identificador único de la publicación.
Publicacion	tituloPublicacion	string	private (-)	Título del producto o servicio.
Publicacion	descripcionPublicacion	string	private (-)	Descripción detallada de la publicación.
Publicacion	precioPublicacion	decimal	private (-)	Precio establecido por el vendedor.
Publicacion	ubicacionPublicacion	string	private (-)	Ubicación de referencia.
Publicacion	telefonoPublicacion	string	private (-)	Número de contacto de la publicación.
Publicacion	estadoPublicacion	EstadoPublicacion	private (-)	Estado actual: activa, pausada o vendida.
Publicacion	fechaRegistroPublicacion	DateTime	private (-)	Fecha y hora de creación.
Categoria	idCategoria	int	private (-)	Identificador único de la categoría.
Categoria	nombreCategoria	string	private (-)	Nombre descriptivo de la categoría.
Categoria	slugCategoria	string	private (-)	Identificador textual utilizado en búsquedas o URL.
Categoria	iconoCategoria	string	private (-)	Icono representativo de la categoría.
ImagenPublicacion	idImagen	long	private (-)	Identificador único de la imagen.
ImagenPublicacion	urlImagen	string	private (-)	Dirección de almacenamiento de la imagen.

Clase	Atributo	Tipo	Visibilidad	Descripción
ImagenPublicacion	ordenImagen	int	private (-)	Orden de presentación.
ImagenPublicacion	portadaImagen	boolean	private (-)	Indica si es la imagen de portada.
Favorito	idFavorito	int	private (-)	Identificador único del favorito.
Favorito	fechaFavorito	DateTime	private (-)	Fecha y hora de registro del favorito.
Contacto	idContacto	long	private (-)	Identificador único del contacto.
Contacto	fechaContacto	DateTime	private (-)	Fecha y hora de la solicitud de contacto.
Calificacion	idCalificacion	long	private (-)	Identificador único de la calificación.
Calificacion	puntuacionCalificacion	int	private (-)	Puntuación asignada entre uno y cinco.
Calificacion	comentarioCalificacion	string	private (-)	Comentario de la evaluación.
Calificacion	fechaCalificacion	DateTime	private (-)	Fecha y hora de emisión.
Reporte	idReporte	long	private (-)	Identificador único del reporte.
Reporte	motivoReporte	MotivoReporte	private (-)	Motivo por el que se reporta la publicación.
Reporte	descripcionReporte	string	private (-)	Descripción adicional del reporte.
Reporte	estadoReporte	EstadoReporte	private (-)	Estado actual del proceso de revisión.
Reporte	fechaReporte	DateTime	private (-)	Fecha y hora de creación del reporte.

Fuente: Elaboración propia.

Tabla 4.4. Enumeraciones utilizadas como tipos de atributos.

Enumeración	Atributo que la utiliza	Valores permitidos
RolUsuario	Usuario.rolUsuario	COMPRADOR, VENDEDOR, ADMINISTRADOR
EstadoPublicacion	Publicacion.estadoPublicacion	ACTIVA, PAUSADA, VENDIDA
EstadoReporte	Reporte.estadoReporte	PENDIENTE, REVISADO, DESESTIMADO
MotivoReporte	Reporte.motivoReporte	PUBLICACION_FALSA, CONTENIDO_INAPROPIADO, PRECIO_ENGANOSO, POSIBLE_ESTAFA, OTRO

Fuente: Elaboración propia.

4.4 Métodos

Los métodos representan el comportamiento de las clases. En el Diagrama de Clases se definieron con visibilidad pública, representada por el símbolo +, para permitir su invocación desde otros componentes del sistema.

Tabla 4.5. Métodos de las clases de UTEQMarket.

Clase	Método y parámetros	Retorno	Visibilidad	Descripción
Usuario	registrarUsuario(nombreUsuario: string, correoUsuario: string, claveUsuario: string)	Usuario	public (+)	Registra una nueva cuenta y devuelve el usuario creado.
Usuario	verificarCorreoUsuario(tokenUsuario: string)	boolean	public (+)	Verifica el correo mediante el token asociado.
Usuario	iniciarSesionUsuario(correoUsuario: string, claveUsuario: string)	boolean	public (+)	Valida las credenciales e inicia la sesión.
Usuario	cerrarSesionUsuario()	void	public (+)	Finaliza la sesión activa.
Usuario	validarClaveUsuario(claveUsuario: string)	boolean	public (+)	Comprueba las reglas de seguridad de la contraseña.
Usuario	actualizarPerfilUsuario(datosUsuario: PerfilUsuarioDTO)	boolean	public (+)	Actualiza los datos del perfil.
Publicacion	crearPublicacion(datosPublicacion: PublicacionDTO)	Publicacion	public (+)	Crea una publicación y devuelve el objeto registrado.
Publicacion	editarPublicacion(datosPublicacion: PublicacionDTO)	boolean	public (+)	Modifica una publicación existente.
Publicacion	cambiarEstadoPublicacion(nuevoEstadoPublicacion: EstadoPublicacion)	boolean	public (+)	Cambia el estado de la publicación.
Publicacion	eliminarPublicacion()	boolean	public (+)	Elimina una publicación cuando existe autorización.
Publicacion	calcularReputacionVendedor()	double	public (+)	Calcula la reputación a partir de las calificaciones.
Publicacion	validarPrecioPublicacion(precioPublicacion: decimal)	boolean	public (+)	Comprueba que el precio sea positivo y válido.
Categoria	crearCategoria(nombreCategoria: string, slugCategoria: string, iconoCategoria: string)	Categoria	public (+)	Registra una categoría.
Categoria	listarPublicacionesCategoria()	List<Publicacion>	public (+)	Devuelve las publicaciones de una categoría.
Categoria	obtenerCategoriaPorSlug(slugCategoria: string)	Categoria	public (+)	Busca una categoría mediante su slug.
ImagenPublicacion	agregarImagen(archivoImagen: ArchivoDTO)	boolean	public (+)	Agrega una imagen después de validar el archivo.
ImagenPublicacion	eliminarImagen()	boolean	public (+)	Elimina una imagen asociada.
ImagenPublicacion	establecerPortadaImagen()	void	public (+)	Establece la imagen seleccionada como portada.
Favorito	agregarFavorito(idUsuarioFavorito: long, idPublicacionFavorito: long)	boolean	public (+)	Registra una publicación como favorita.
Favorito	retirarFavorito()	boolean	public (+)	Retira una publicación de favoritos.
Contacto	registrarContacto(idUsuarioContacto: long, idPublicacion: long)	boolean	public (+)	Registra una solicitud de contacto.
Contacto	existeContactoPrevio(idUsuario: long, idPublicacion: long)	boolean	public (+)	Comprueba si ya existe un contacto previo.

Clase	Método y parámetros	Retorno	Visibilidad	Descripción
Calificacion	crearCalificacion(datosCalificacion: CalificacionDTO)	boolean	public (+)	Registra una puntuación y comentario.
Calificacion	validarElegibilidadCalificacion(idUsuarioCalificacion: long, idPublicacionCalificacion: long)	boolean	public (+)	Verifica que el usuario pueda calificar.
Calificacion	recalcularReputacionVendedor(idVendedor: long)	void	public (+)	Actualiza la reputación del vendedor.
Reporte	crearReporte(datosReporte: ReporteDTO)	boolean	public (+)	Registra un reporte sobre una publicación.
Reporte	marcarRevisadoReporte()	void	public (+)	Cambia el estado del reporte a revisado.
Reporte	desestimarReporte()	void	public (+)	Cambia el estado a desestimado.
Reporte	ocultarPublicacionReportada()	boolean	public (+)	Oculta una publicación que incumple las reglas.

Fuente: Elaboración propia.

Tabla 4.6. Significado de los tipos de retorno.

Tipo de retorno	Significado
boolean	Devuelve true cuando la operación se completa y false cuando no puede completarse.
void	Ejecuta una acción sin devolver un valor.
Usuario	Devuelve un objeto de la clase Usuario.
Publicacion	Devuelve un objeto de la clase Publicacion.
Categoria	Devuelve un objeto de la clase Categoria.
List<Publicacion>	Devuelve una colección de publicaciones.
double	Devuelve un valor decimal utilizado para la reputación.

Fuente: Elaboración propia.

4.5 Relaciones

Las relaciones UML describen la manera en que las clases se conectan y colaboran. En UTEQMarket se emplean asociaciones y una composición. La agregación, la dependencia explícita y la generalización no forman parte del diagrama final.

Asociación

La asociación representa un vínculo estructural entre clases. En el diagrama se muestra mediante una línea simple sin punta de flecha, acompañada por el nombre de la relación y sus multiplicidades.

Tabla 4.7. Asociaciones del Diagrama de Clases.

Clase A	Relación	Clase B	Descripción
Usuario	publica	Publicacion	Un usuario puede registrar varias publicaciones; cada publicación pertenece a un usuario.
Categoria	clasifica	Publicacion	Una categoría puede clasificar varias publicaciones; cada publicación pertenece a una categoría.
Usuario	marca	Favorito	Un usuario puede generar varios registros de favorito.
Publicacion	es marcada	Favorito	Una publicación puede aparecer en varios registros de favorito.
Usuario	contacta	Contacto	Un usuario puede generar varias solicitudes de contacto.
Publicacion	recibe contacto	Contacto	Una publicación puede recibir varias solicitudes de contacto.
Usuario	califica	Calificacion	Un usuario puede emitir varias calificaciones.
Publicacion	recibe calificación	Calificacion	Una publicación puede recibir varias calificaciones.
Usuario	reporta	Reporte	Un usuario puede generar varios reportes.
Publicacion	es reportada	Reporte	Una publicación puede recibir varios reportes.

Fuente: Elaboración propia.

Composición

La composición representa una relación fuerte entre un objeto principal y sus partes. En UTEQMarket se utiliza entre Publicacion e ImagenPublicacion. El rombo negro se ubica junto a Publicacion, porque una imagen no posee sentido independiente dentro del sistema.

◆ Publicacion 1..1 — contiene — 0..* ImagenPublicacion

Agregación

El modelo no utiliza agregación. Esta relación se emplearía cuando una parte pudiera existir independientemente del objeto contenedor. En este caso, ImagenPublicacion depende de Publicacion, por lo que corresponde utilizar composición.

Dependencia

No se dibujaron dependencias explícitas. Algunos métodos utilizan objetos auxiliares como PerfilUsuarioDTO, PublicacionDTO, ArchivoDTO, CalificacionDTO y ReporteDTO; esto representa una dependencia conceptual, pero dichas clases no forman parte del alcance del diagrama.

Generalización

No se utiliza generalización o herencia. Los roles comprador, vendedor y administrador no se modelan como subclases de Usuario, sino mediante el atributo rolUsuario de tipo RolUsuario. Esta decisión evita duplicación de datos y comportamiento.

Nota técnica. En el modelo físico, los nombres publica, clasifica, marca, contacta, califica y reporta no se convierten en tablas. Se implementan mediante referencias y claves foráneas entre las ocho tablas principales.

4.6 Cardinalidades

Las cardinalidades o multiplicidades indican la cantidad mínima y máxima de instancias que pueden participar en una relación. PowerDesigner utiliza la notación 1..1 para representar una instancia obligatoria y 0..* para representar cero o muchas instancias.

Tabla 4.8. Significado de las cardinalidades UML.

Cardinalidad	Significado	Interpretación
1 o 1..1	Exactamente uno	La relación es obligatoria y solo admite una instancia.
0..1	Cero o uno	La relación es opcional y admite como máximo una instancia.
1..*	Uno o muchos	Debe existir al menos una instancia y pueden existir varias.
0..*	Cero o muchos	La relación es opcional y puede contener múltiples instancias.
*	Muchos	Cantidad indeterminada; se recomienda especificar el mínimo.

Fuente: Elaboración propia.

Tabla 4.9. Cardinalidades del Diagrama de Clases de UTEQMarket.

Clase A	Relación	Clase B	Cardinalidad	Interpretación
Usuario	publica	Publicacion	1..1 — 0..*	Un usuario puede registrar cero o muchas publicaciones; cada publicación pertenece a un usuario.
Categoria	clasifica	Publicacion	1..1 — 0..*	Una categoría puede clasificar cero o muchas publicaciones; cada publicación pertenece a una categoría.
Publicacion	contiene	ImagenPublicacion	1..1 — 0..*	Una publicación puede contener cero o muchas imágenes; cada imagen pertenece a una publicación.
Usuario	marca	Favorito	1..1 — 0..*	Un usuario puede generar cero o muchos favoritos; cada favorito pertenece a un usuario.
Publicacion	es marcada	Favorito	1..1 — 0..*	Una publicación puede aparecer en cero o muchos favoritos; cada favorito corresponde a una publicación.
Usuario	contacta	Contacto	1..1 — 0..*	Un usuario puede realizar cero o muchos contactos; cada contacto pertenece a un usuario.
Publicacion	recibe contacto	Contacto	1..1 — 0..*	Una publicación puede recibir cero o muchos contactos; cada contacto corresponde a una publicación.
Usuario	califica	Calificacion	1..1 — 0..*	Un usuario puede emitir cero o muchas calificaciones; cada calificación pertenece a un usuario.
Publicacion	recibe calificación	Calificacion	1..1 — 0..*	Una publicación puede recibir cero o muchas calificaciones; cada calificación corresponde a una publicación.
Usuario	reporta	Reporte	1..1 — 0..*	Un usuario puede generar cero o muchos reportes; cada reporte pertenece a un usuario.
Publicacion	es reportada	Reporte	1..1 — 0..*	Una publicación puede recibir cero o muchos reportes; cada reporte corresponde a una publicación.

Fuente: Elaboración propia.

Las multiplicidades 0..1 y 1..* se explican como parte de la notación UML, pero no se utilizan en las once relaciones finales del modelo. En UTEQMarket, el extremo principal se configuró como 1..1 y el extremo dependiente como 0..*.

4.7 Estereotipos utilizados

Los estereotipos amplían el significado de los elementos UML e indican la función que cumple cada clase dentro del modelo. En UTEQMarket se utilizaron los estereotipos <<Entity>> y <<enumeration>>.

Tabla 4.10. Estereotipos utilizados en UTEQMarket.

Estereotipo	Elementos que lo utilizan	Descripción
<<Entity>>	Usuario, Publicacion, Categoria, ImagenPublicacion, Favorito, Contacto, Calificacion y Reporte	Identifica clases persistentes que representan información y comportamiento del dominio.
<<enumeration>>	RolUsuario, EstadoPublicacion, EstadoReporte y MotivoReporte	Identifica conjuntos cerrados de valores permitidos.

Fuente: Elaboración propia.

Estereotipo <<Entity>>

El estereotipo <<Entity>> se asignó a las ocho clases principales porque representan información persistente del dominio. Las clases Favorito, Contacto, Calificacion y Reporte también son entidades, ya que almacenan datos propios de las interacciones.

Estereotipo <<enumeration>>

El estereotipo <<enumeration>> se asignó a RolUsuario, EstadoPublicacion, EstadoReporte y MotivoReporte. Estas enumeraciones permanecen como elementos independientes en el OOM y se utilizan como tipos de atributos, sin asociaciones ni cardinalidades.

Estereotipos no utilizados

Los estereotipos <<Controller>> y <<Boundary>> no se incorporaron porque el diagrama se concentra en el modelo de dominio. Agregarlos sin clases de control o interfaz reales introduciría elementos que no fueron definidos en los requisitos.

Nota técnica. En el modelo físico de SQL Server, las enumeraciones no se generan como tablas. Se implementan mediante columnas VARCHAR y reglas de validación, mientras que en el OOM se conservan como <<enumeration>>.

4.8 Validación del modelo

La validación se realizó en dos niveles. Primero se comprobó la consistencia del Modelo Orientado a Objetos mediante Check Model de PowerDesigner. Posteriormente se generó y validó el Modelo Físico de Datos, y el script SQL fue ejecutado en Microsoft SQL Server para verificar la creación de tablas, claves primarias, claves foráneas y relaciones.

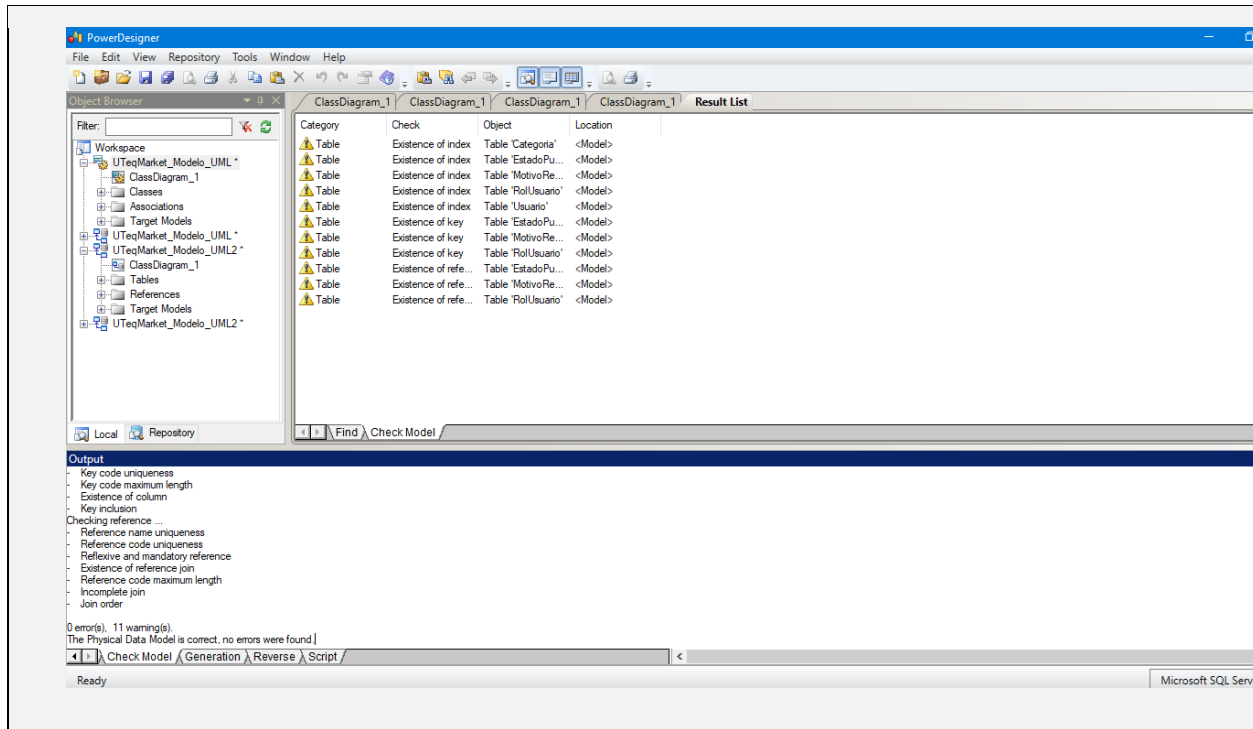


Figura 4.2. Resultado de la validación del Modelo Orientado a Objetos de UTEQMarket.

Fuente: Elaboración propia mediante PowerDesigner

Como resultado de la comprobación del OOM, PowerDesigner no registró errores críticos. Las observaciones mostradas por la herramienta fueron revisadas para comprobar los nombres, códigos, atributos, operaciones, estereotipos y multiplicidades del modelo.

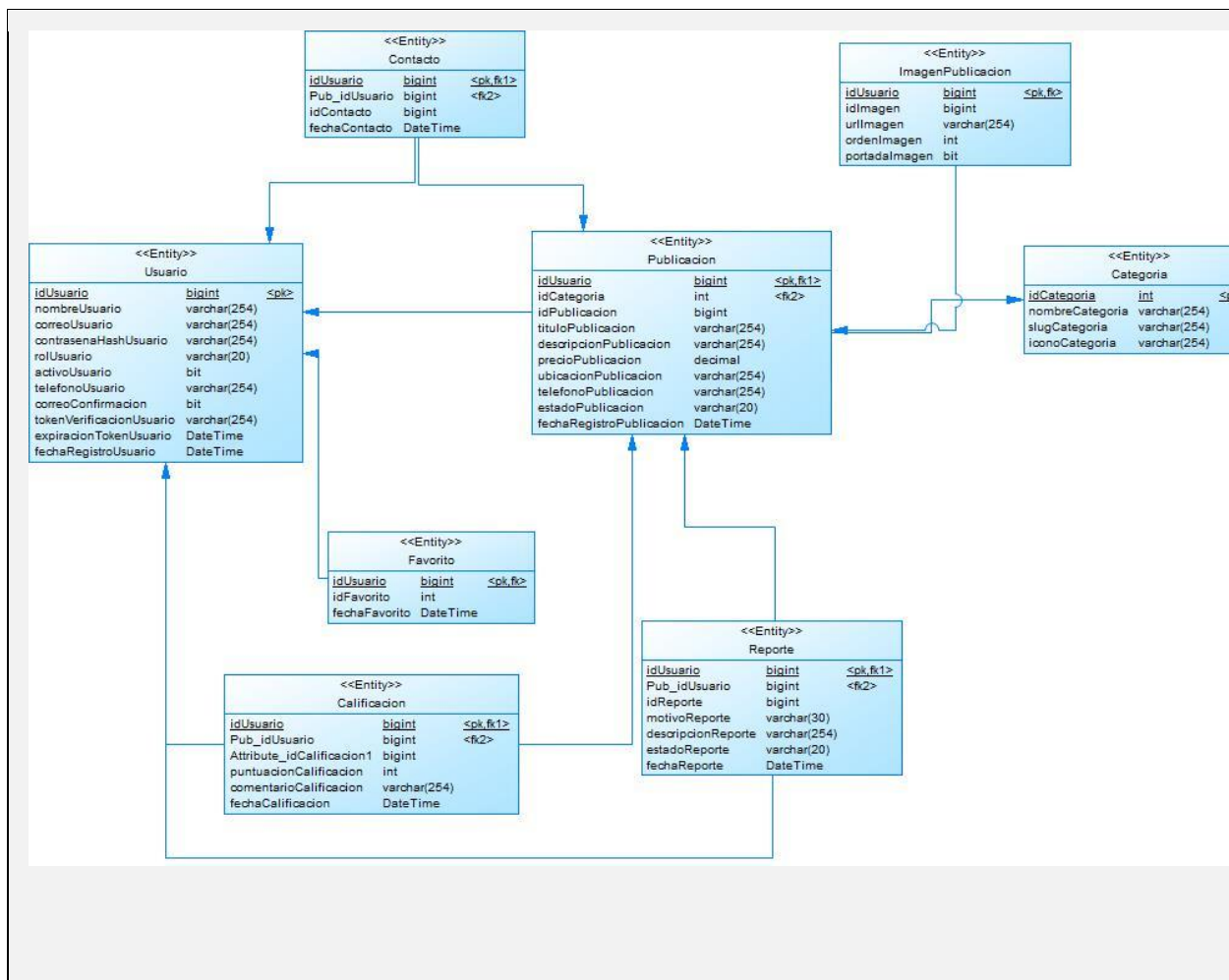


Figura 4.3. Modelo Físico de Datos final de UTEQMarket en PowerDesigner.

Fuente: Elaboración propia mediante PowerDesigner

En el PDM se configuraron como claves primarias `idUsuario`, `idCategoria`, `idPublicacion`, `idImagen`, `idFavorito`, `idContacto`, `idCalificacion` e `idReporte`. Las asociaciones del OOM se transformaron en referencias físicas y claves foráneas. Las enumeraciones se conservaron en el modelo UML, pero en la base de datos se implementaron como columnas de texto controladas.

Tabla 4.11. Correspondencia entre asociaciones UML y claves foráneas.

Tabla principal	Tabla dependiente	Clave foránea
Usuario	Publicacion	<code>idUsuarioPublicacion</code>
Categoria	Publicacion	<code>idCategoriaPublicacion</code>
Publicacion	ImagenPublicacion	<code>idPublicacionImagen</code>
Usuario	Favorito	<code>idUsuarioFavorito</code>
Publicacion	Favorito	<code>idPublicacionFavorito</code>

Tabla principal	Tabla dependiente	Clave foránea
Usuario	Contacto	idUsuarioContacto
Publicacion	Contacto	idPublicacionContacto
Usuario	Calificacion	idUsuarioCalificacion
Publicacion	Calificacion	idPublicacionCalificacion
Usuario	Reporte	idUsuarioReporte
Publicacion	Reporte	idPublicacionReporte

Fuente: Elaboración propia.

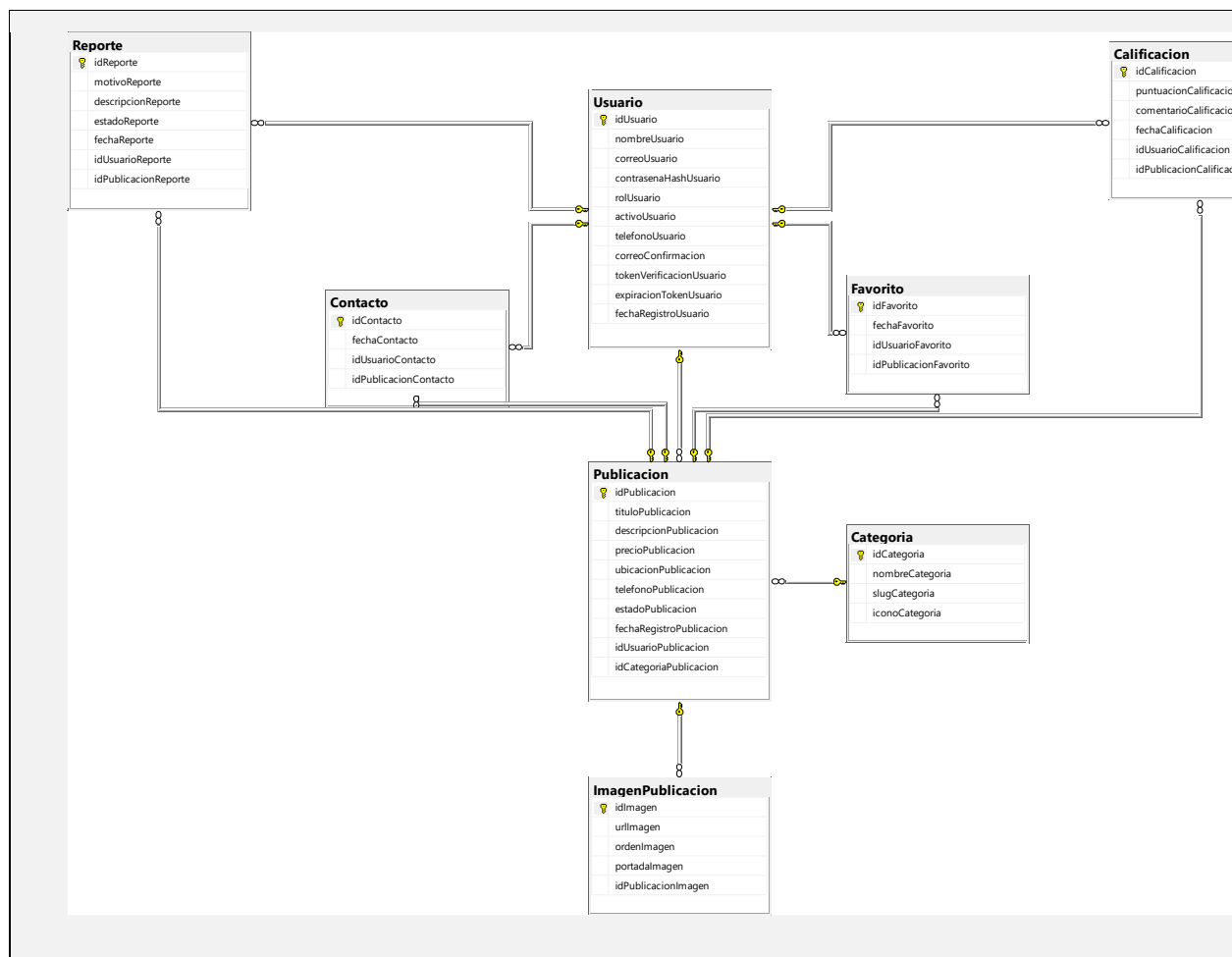


Figura 4.4. Implementación y relaciones de UTEQMarket en Microsoft SQL Server.

Fuente: Elaboración propia mediante SQL Server

La ejecución del script SQL permitió crear correctamente las ocho tablas principales y sus restricciones de integridad referencial. El diagrama de SQL Server confirmó visualmente que las

tablas se encuentran conectadas mediante claves foráneas y que las asociaciones no fueron transformadas en tablas innecesarias.

En conclusión, el Diagrama de Clases, el Modelo Físico de Datos y la implementación en SQL Server presentan correspondencia estructural. El modelo se considera válido y puede utilizarse como base para las siguientes etapas de diseño e implementación de UTEQMarket.

Código SQL final para Microsoft SQL Server

```
/* =====
UTeqMarket - Modelo físico final para Microsoft SQL Server
El modelo contiene únicamente ocho tablas principales.
===== */

USE master;
GO

/* Crear la base de datos solamente si todavía no existe */
IF DB_ID(N'UTeqMarket_Modelo_UML') IS NULL
BEGIN
    CREATE DATABASE UTeqMarket_Modelo_UML;
END;
GO

USE UTeqMarket_Modelo_UML;
GO

/* =====
Eliminar las tablas anteriores en orden de dependencia
===== */

IF OBJECT_ID(N'dbo.Reporte', N'U') IS NOT NULL
    DROP TABLE dbo.Reporte;
GO

IF OBJECT_ID(N'dbo.Calificacion', N'U') IS NOT NULL
    DROP TABLE dbo.Calificacion;
GO

IF OBJECT_ID(N'dbo.Contacto', N'U') IS NOT NULL
    DROP TABLE dbo.Contacto;
GO

IF OBJECT_ID(N'dbo.Favorito', N'U') IS NOT NULL
    DROP TABLE dbo.Favorito;
GO

IF OBJECT_ID(N'dbo.ImagenPublicacion', N'U') IS NOT NULL
    DROP TABLE dbo.ImagenPublicacion;
GO

IF OBJECT_ID(N'dbo.Publicacion', N'U') IS NOT NULL
```

```

    DROP TABLE dbo.Publicacion;
GO

IF OBJECT_ID(N'dbo.Categoria', N'U') IS NOT NULL
    DROP TABLE dbo.Categoria;
GO

IF OBJECT_ID(N'dbo.Usuario', N'U') IS NOT NULL
    DROP TABLE dbo.Usuario;
GO

/* =====
   Tabla: Usuario
   ===== */

CREATE TABLE dbo.Usuario
(
    idUsuario                BIGINT NOT NULL,
    nombreUsuario            VARCHAR(254) NULL,
    correoUsuario            VARCHAR(254) NULL,
    contrasenaHashUsuario    VARCHAR(254) NULL,
    rolUsuario               VARCHAR(20) NULL,
    activoUsuario            BIT NULL,
    telefonoUsuario          VARCHAR(254) NULL,
    correoConfirmacion       BIT NULL,
    tokenVerificacionUsuario VARCHAR(254) NULL,
    expiracionTokenUsuario   DATETIME NULL,
    fechaRegistroUsuario     DATETIME NULL,

    CONSTRAINT PK_Usuario
        PRIMARY KEY (idUsuario),

    CONSTRAINT CK_Usuario_Rol
        CHECK
        (
            rolUsuario IS NULL
            OR rolUsuario IN
            (
                'COMPRADOR',
                'VENDEDOR',
                'ADMINISTRADOR'
            )
        )
);
GO

/* =====
   Tabla: Categoria
   ===== */

CREATE TABLE dbo.Categoria
(
    idCategoria    INT NOT NULL,
    nombreCategoria VARCHAR(254) NULL,
    slugCategoria  VARCHAR(254) NULL,
    iconoCategoria VARCHAR(254) NULL,

    CONSTRAINT PK_Categoria

```

```

        PRIMARY KEY (idCategoria)
    );
GO

/* =====
   Tabla: Publicacion
   ===== */

CREATE TABLE dbo.Publicacion
(
    idPublicacion          BIGINT NOT NULL,
    tituloPublicacion      VARCHAR(254) NULL,
    descripcionPublicacion VARCHAR(254) NULL,
    precioPublicacion      DECIMAL(10,2) NULL,
    ubicacionPublicacion   VARCHAR(254) NULL,
    telefonoPublicacion    VARCHAR(254) NULL,
    estadoPublicacion      VARCHAR(20) NULL,
    fechaRegistroPublicacion DATETIME NULL,

    idUsuarioPublicacion   BIGINT NOT NULL,
    idCategoriaPublicacion INT NOT NULL,

    CONSTRAINT PK_Publicacion
        PRIMARY KEY (idPublicacion),

    CONSTRAINT CK_Publicacion_Estado
        CHECK
        (
            estadoPublicacion IS NULL
            OR estadoPublicacion IN
            (
                'ACTIVA',
                'PAUSADA',
                'VENDIDA'
            )
        ),

    CONSTRAINT FK_Publicacion_Usuario
        FOREIGN KEY (idUsuarioPublicacion)
        REFERENCES dbo.Usuario (idUsuario),

    CONSTRAINT FK_Publicacion_Categoria
        FOREIGN KEY (idCategoriaPublicacion)
        REFERENCES dbo.Categoria (idCategoria)
);
GO

/* =====
   Tabla: ImagenPublicacion
   ===== */

CREATE TABLE dbo.ImagenPublicacion
(
    idImagen          BIGINT NOT NULL,
    urlImagen         VARCHAR(254) NULL,
    ordenImagen       INT NULL,
    portadaImagen     BIT NULL,

```

```

        idPublicacionImagen          BIGINT NOT NULL,

        CONSTRAINT PK_ImagenPublicacion
            PRIMARY KEY (idImagen),

        CONSTRAINT FK_ImagenPublicacion_Publicacion
            FOREIGN KEY (idPublicacionImagen)
            REFERENCES dbo.Publicacion (idPublicacion)
    );
GO

/* =====
   Tabla: Favorito
   ===== */

CREATE TABLE dbo.Favorito
(
    idFavorito                      INT NOT NULL,
    fechaFavorito                  DATETIME NULL,

    idUsuarioFavorito              BIGINT NOT NULL,
    idPublicacionFavorito          BIGINT NOT NULL,

    CONSTRAINT PK_Favorito
        PRIMARY KEY (idFavorito),

    CONSTRAINT FK_Favorito_Usuario
        FOREIGN KEY (idUsuarioFavorito)
        REFERENCES dbo.Usuario (idUsuario),

    CONSTRAINT FK_Favorito_Publicacion
        FOREIGN KEY (idPublicacionFavorito)
        REFERENCES dbo.Publicacion (idPublicacion)
);
GO

/* =====
   Tabla: Contacto
   ===== */

CREATE TABLE dbo.Contacto
(
    idContacto                    BIGINT NOT NULL,
    fechaContacto                 DATETIME NULL,

    idUsuarioContacto             BIGINT NOT NULL,
    idPublicacionContacto         BIGINT NOT NULL,

    CONSTRAINT PK_Contacto
        PRIMARY KEY (idContacto),

    CONSTRAINT FK_Contacto_Usuario
        FOREIGN KEY (idUsuarioContacto)
        REFERENCES dbo.Usuario (idUsuario),

    CONSTRAINT FK_Contacto_Publicacion
        FOREIGN KEY (idPublicacionContacto)
        REFERENCES dbo.Publicacion (idPublicacion)
);
GO

```

```

);
GO

/* =====
Tabla: Calificacion
===== */

CREATE TABLE dbo.Calificacion
(
    idCalificacion          BIGINT NOT NULL,
    puntuacionCalificacion  INT NULL,
    comentarioCalificacion  VARCHAR(254) NULL,
    fechaCalificacion       DATETIME NULL,

    idUsuarioCalificacion   BIGINT NOT NULL,
    idPublicacionCalificacion BIGINT NOT NULL,

    CONSTRAINT PK_Calificacion
        PRIMARY KEY (idCalificacion),

    CONSTRAINT FK_Calificacion_Usuario
        FOREIGN KEY (idUsuarioCalificacion)
        REFERENCES dbo.Usuario (idUsuario),

    CONSTRAINT FK_Calificacion_Publicacion
        FOREIGN KEY (idPublicacionCalificacion)
        REFERENCES dbo.Publicacion (idPublicacion)
);
GO

/* =====
Tabla: Reporte
===== */

CREATE TABLE dbo.Reporte
(
    idReporte          BIGINT NOT NULL,
    motivoReporte       VARCHAR(30) NULL,
    descripcionReporte  VARCHAR(254) NULL,
    estadoReporte       VARCHAR(20) NULL,
    fechaReporte        DATETIME NULL,

    idUsuarioReporte    BIGINT NOT NULL,
    idPublicacionReporte BIGINT NOT NULL,

    CONSTRAINT PK_Reporte
        PRIMARY KEY (idReporte),

    CONSTRAINT CK_Reporte_Motivo
        CHECK
        (
            motivoReporte IS NULL
            OR motivoReporte IN
            (
                'PUBLICACION_FALSA',
                'CONTENIDO_INAPROPIADO',
                'PRECIO_ENGANOSO',
                'POSIBLE_ESTAFA',
            )
        )
);
GO

```



```

        'OTRO'
    )
),
CONSTRAINT CK_Reporte_Estado
CHECK
(
    estadoReporte IS NULL
    OR estadoReporte IN
    (
        'PENDIENTE',
        'REVISADO',
        'DESESTIMADO'
    )
),

CONSTRAINT FK_Reporte_Usuario
FOREIGN KEY (idUsuarioReporte)
REFERENCES dbo.Usuario (idUsuario),

CONSTRAINT FK_Reporte_Publicacion
FOREIGN KEY (idPublicacionReporte)
REFERENCES dbo.Publicacion (idPublicacion)
);
GO

PRINT 'La base de datos UTEqMarket_Modelo_UML fue creada correctamente.';
PRINT 'Tablas creadas: Usuario, Categoria, Publicacion, ImagenPublicacion, Favorito,
Contacto, Calificacion y Reporte.';
GO

```

5. Configuración del generador

La configuración del generador constituye la etapa en la que el Modelo Orientado a Objetos se prepara para la transformación Modelo a Texto. En esta práctica se utilizó el archivo OOM de UTEQMarket y se seleccionó C# 2.0 como lenguaje objetivo. La configuración permitió generar un archivo .cs por cada clase seleccionada, conservando los nombres, atributos y operaciones definidos en el modelo.

5.1 Configuración de PowerDesigner

La generación se ejecutó desde el Modelo Orientado a Objetos UTEqMarket_Modelo_UML.oom, identificado en PowerDesigner por las carpetas Classes, Associations y Target Models. Esta distinción es importante porque el Modelo Físico de Datos, que contiene Tables y References, se utiliza para generar SQL y no código de clases C#.

Antes de generar el código se cambió el lenguaje del modelo mediante la opción Language → Change Current Object Language. En la lista de lenguajes disponibles se seleccionó C# 2.0, evitando las definiciones marcadas como obsoletas. Posteriormente se abrió la ventana Generation, se definió el directorio de salida y se mantuvo activa la comprobación Check model.

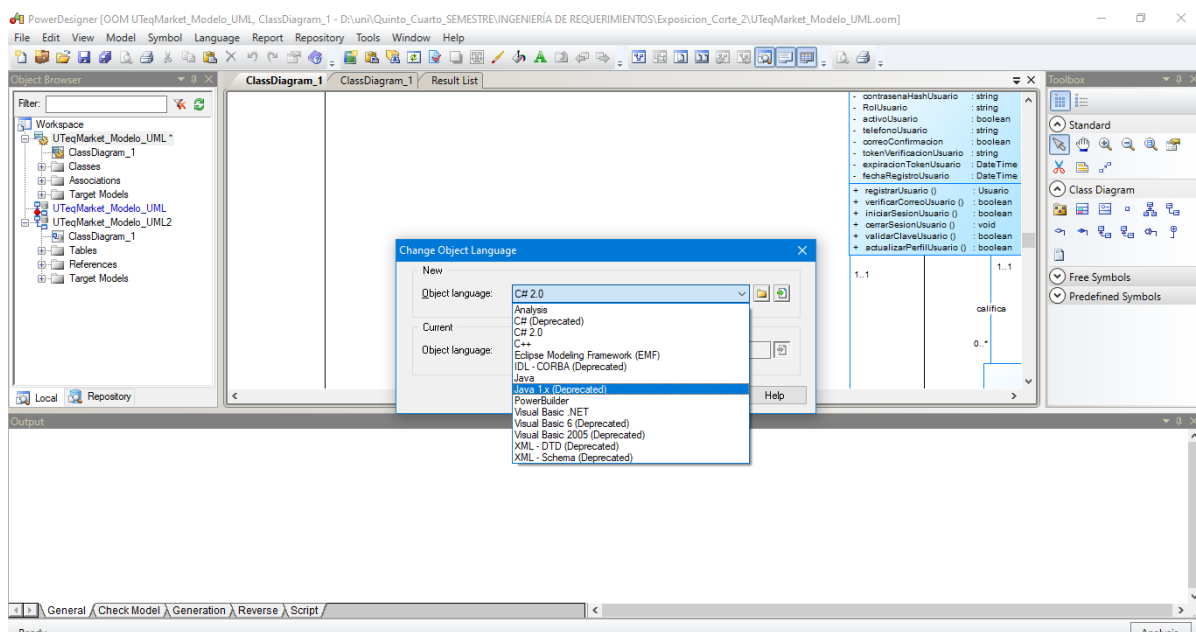


Figura 5.1. Selección de C# 2.0 como lenguaje objetivo en PowerDesigner.

Fuente: Elaboración propia mediante PowerDesigner.

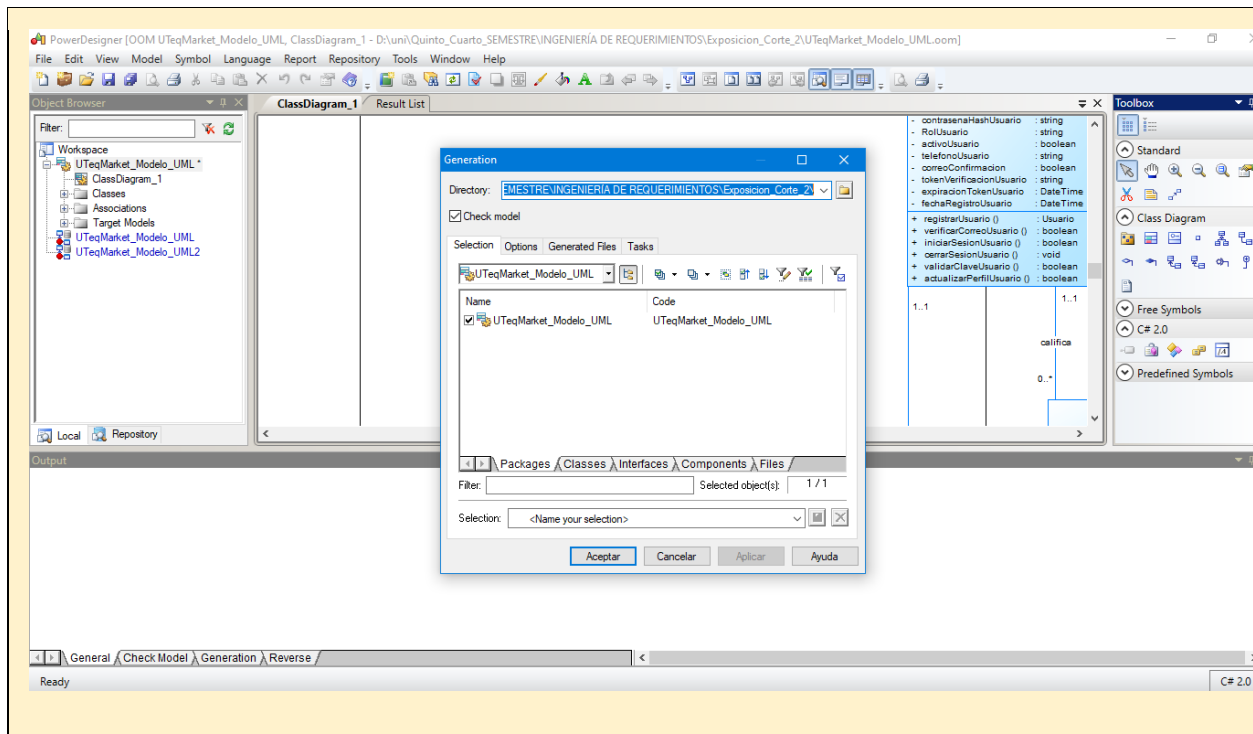


Figura 5.2. Configuración del directorio y selección de objetos para la generación de C#.

Fuente: Elaboración propia mediante PowerDesigner

5.2 Lenguaje objetivo

El lenguaje objetivo utilizado fue C# 2.0. Esta definición de lenguaje proporciona a PowerDesigner las reglas necesarias para transformar clases, atributos y operaciones UML en estructuras de código C#. La elección de C# responde a que el proyecto puede integrarse posteriormente con el ecosistema .NET y con Visual Studio.

Elemento	Configuración aplicada
Modelo de origen	Modelo Orientado a Objetos (.oom)
Lenguaje objetivo	C# 2.0
Extensión generada	.cs
Entorno de verificación	Microsoft Visual Studio
Tipo de proyecto de prueba	Biblioteca de clases de C#

5.3 Opciones del generador

Las opciones del generador controlan la manera en que los elementos del modelo se convierten en archivos fuente. Para esta práctica se utilizaron las plantillas predeterminadas incluidas en la definición de C# 2.0 de PowerDesigner.

Opción	Descripción
Nombres	El nombre de cada archivo se obtuvo del Code de la clase UML. Por ejemplo, la clase Usuario generó Usuario.cs y Publicacion generó Publicacion.cs.
Namespace	El código observado fue generado sin una declaración namespace explícita. Los archivos se incorporaron posteriormente al proyecto UTeqMarket.Modelo de Visual Studio. En una versión posterior puede añadirse el namespace UTeqMarket.Modelo o configurarse mediante paquetes UML.
Sobrescribir	Para la primera generación se utilizó una carpeta destinada exclusivamente al código generado. En una regeneración se debe conservar una copia previa, porque los archivos con el mismo nombre pueden ser sustituidos.
Plantillas	Se emplearon las plantillas predeterminadas de C# 2.0. Estas generaron declaraciones de clases, campos privados, firmas de métodos y cuerpos con NotImplementedException.
Validación	Se mantuvo activa la opción Check model para comprobar el OOM antes de iniciar la generación.
Directorio	Los archivos fueron organizados en la carpeta codigo_generado dentro del directorio de la exposición.

5.4 Mapeo UML → Código

La Tabla 5.2 presenta la correspondencia entre los elementos UML y el código obtenido. El resultado real mostró que PowerDesigner generó campos privados en lugar de propiedades automáticas, y que las asociaciones no se materializaron como referencias porque no se configuraron roles navegables para producir colecciones o campos relacionados.

Elemento UML	Representación en C#	Resultado observado
Clase	public class	Cada clase UML generó una clase pública en un archivo independiente.
Atributo	private field	Los atributos privados se generaron como campos privados, por ejemplo private long idUsuario;.
Operación	public method	Las operaciones generaron métodos públicos con la firma correspondiente y un cuerpo inicial con NotImplementedException.
Asociación	Referencia o colección	No se generó automáticamente en los archivos observados. Requiere navegabilidad, nombres de rol o implementación manual.

Elemento UML	Representación en C#	Resultado observado
Generalización	Herencia con : ClaseBase	No se aplicó porque el modelo de UTEQMarket no utiliza generalización.
Enumeración	enum o tipo referenciado	Los archivos generados hacen referencia a EstadoPublicacion, MotivoReporte y EstadoReporte, pero los archivos enum no se generaron en la primera ejecución.

6. Generación del código

6.1 Ejecución

La generación se ejecutó mediante la opción de generación de C# disponible en el menú Language. PowerDesigner procesó las clases seleccionadas y mostró la ventana Generated Files con las rutas de los archivos creados. El panel Output también registró el procesamiento individual de las clases Favorito, ImagenPublicacion, Publicacion, Reporte y Usuario, entre otras.

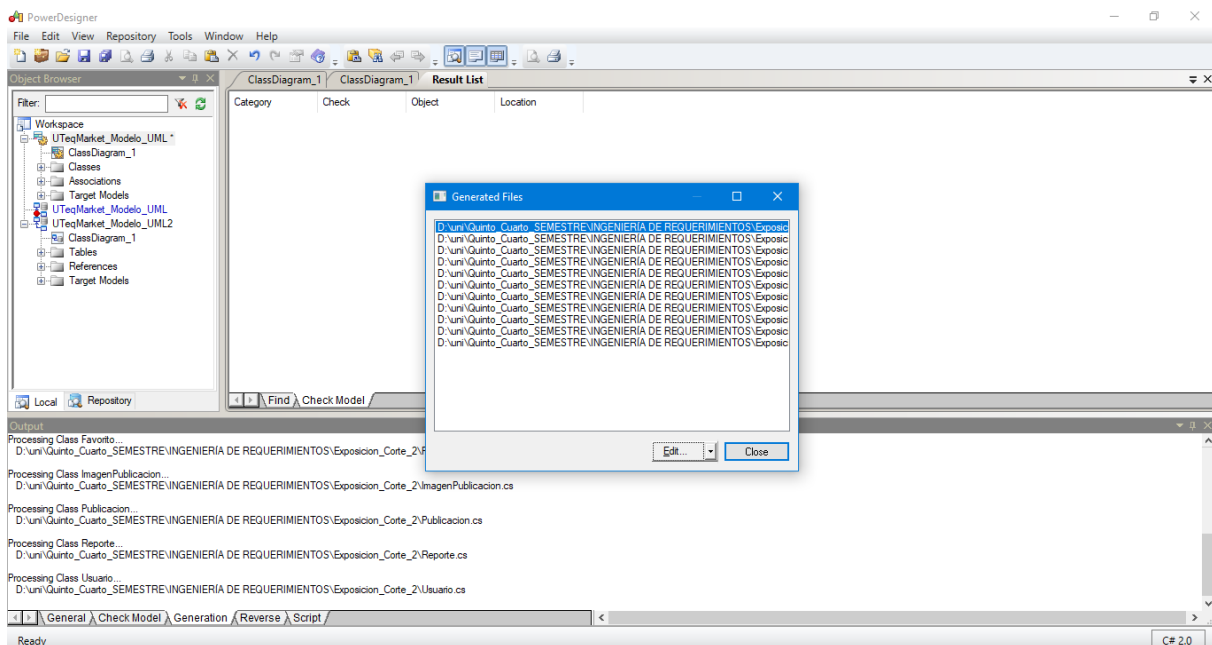


Figura 6.1. Ejecución y archivos generados automáticamente por PowerDesigner.

Fuente: Elaboración propia mediante PowerDesigner.

6.2 Archivos generados

PowerDesigner generó ocho archivos fuente correspondientes a las ocho clases principales del dominio. Los archivos fueron organizados en la carpeta codigo_generado.

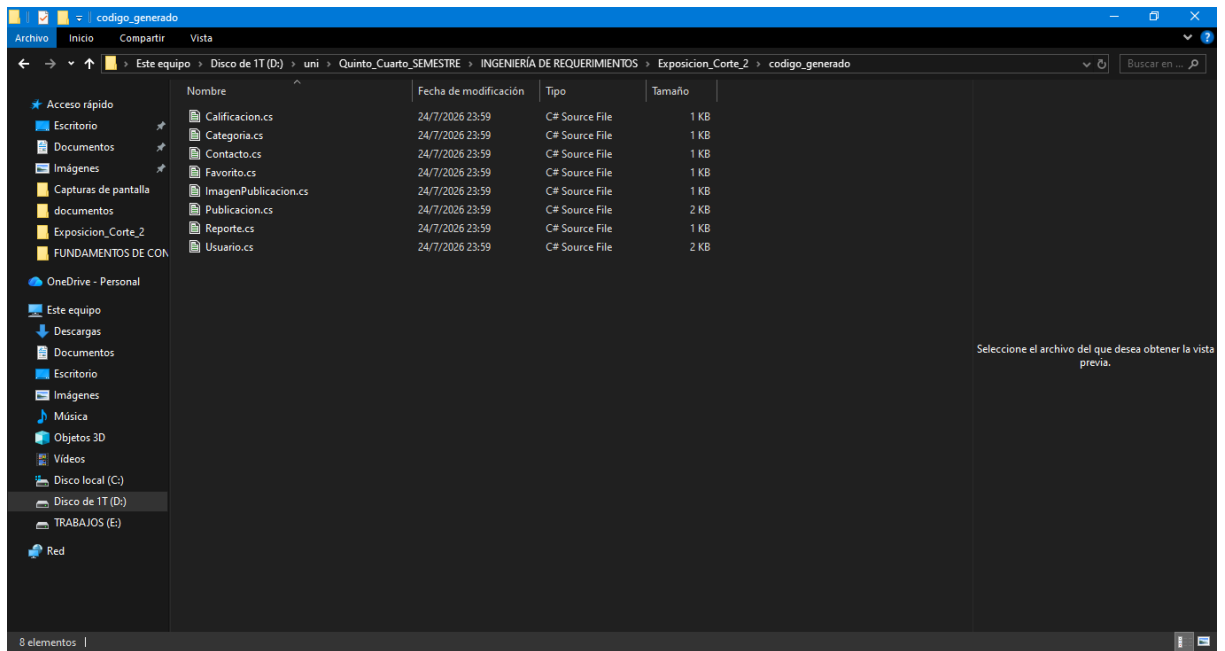


Figura 6.2. Archivos C# generados a partir del Diagrama de Clases.

Fuente: Elaboración propia mediante el Explorador de archivos de Microsoft Windows.

```

codigo_generado
├── Calificacion.cs
├── Categoria.cs
├── Contacto.cs
├── Favorito.cs
├── ImagenPublicacion.cs
├── Publicacion.cs
├── Reporte.cs
└── Usuario.cs

```

No se generaron carpetas Controllers o Services porque el modelo elaborado contiene únicamente entidades del dominio y enumeraciones. Para producir dichos componentes sería necesario modelar clases adicionales con responsabilidades de control y servicio.

6.3 Explicación de los archivos generados

Archivo	Contenido generado
Usuario.cs	Define la clase Usuario, sus campos de cuenta y los métodos de registro, verificación, autenticación y actualización del perfil.

Archivo	Contenido generado
Publicacion.cs	Define la clase Publicacion, sus datos principales y los métodos de creación, edición, cambio de estado, eliminación, reputación y validación del precio.
Categoria.cs	Define la clase Categoria y las operaciones para crear, listar publicaciones y buscar mediante slug.
ImagenPublicacion.cs	Define la clase ImagenPublicacion y las operaciones para agregar, eliminar y establecer la portada.
Favorito.cs	Define la clase Favorito y las operaciones para agregar y retirar una publicación de favoritos.
Contacto.cs	Define la clase Contacto y las operaciones para registrar y comprobar contactos previos.
Calificacion.cs	Define la clase Calificacion y las operaciones de creación, validación de elegibilidad y recálculo de reputación.
Reporte.cs	Define la clase Reporte y las operaciones para crear, revisar, desestimar y ocultar una publicación reportada.

6.4 Correspondencia UML–Código

Clase UML	Archivo generado
Usuario	Usuario.cs
Publicacion	Publicacion.cs
Categoria	Categoria.cs
ImagenPublicacion	ImagenPublicacion.cs
Favorito	Favorito.cs
Contacto	Contacto.cs
Calificacion	Calificacion.cs
Reporte	Reporte.cs

6.5 Limitaciones de la generación automática

La generación automática produjo una estructura inicial útil, pero no una implementación funcional completa. Los archivos conservan los campos y firmas de métodos definidos en el modelo; sin embargo, presentan aspectos que deben completarse manualmente antes de compilar y ejecutar.

Limitación	Acción necesaria
Tipo boolean	El generador produjo boolean, pero C# utiliza la palabra clave bool. Este fue el origen de cuatro errores observados en la primera compilación.
Métodos sin implementación	Los métodos contienen throw new NotImplementedException(); y necesitan lógica de negocio real.
Enumeraciones faltantes	Publicacion.cs y Reporte.cs hacen referencia a EstadoPublicacion, MotivoReporte y EstadoReporte, pero sus archivos enum no fueron generados.
Colecciones	Categoria.cs utiliza List<Publicacion> y requiere using System.Collections.Generic;.
Nombre de atributo	Calificacion.cs conserva attribute_idCalificacion1 en lugar de idCalificacion, lo cual debe corregirse mediante Round Trip.
Asociaciones	No se generaron campos de navegación o colecciones para las asociaciones del diagrama.
Namespace	Los archivos se generaron sin namespace explícito.
Capas adicionales	No se generaron controladores, servicios, repositorios, interfaz gráfica ni acceso a datos porque estos elementos no fueron modelados.

7. Verificación

7.1 Compilación

Los ocho archivos se incorporaron a un proyecto de biblioteca de clases denominado UTeqMarket.Modelo en Visual Studio. La primera compilación permitió identificar cuatro errores CS0246 debido al uso del tipo boolean, que no es una palabra clave válida en C#. La advertencia CS8618 relacionada con comentarioCalificacion corresponde al análisis de nulabilidad y no impide por sí sola la compilación.

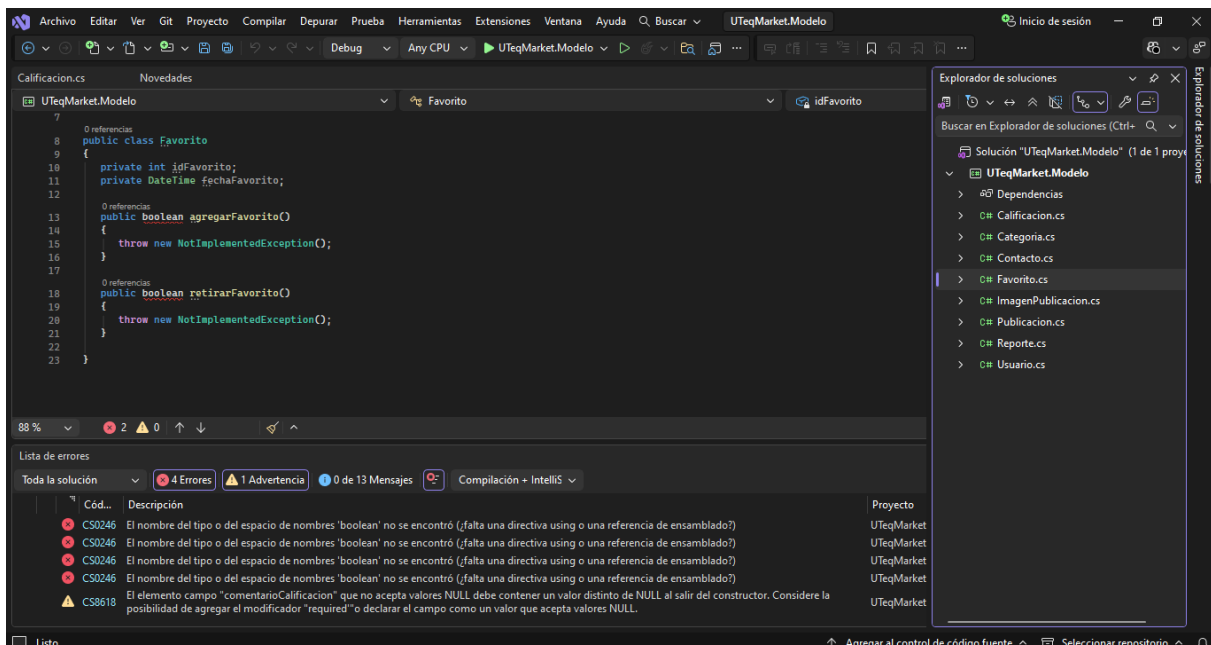


Figura 7.1. Primera compilación del código generado y errores de tipo boolean.

Fuente: Elaboración propia mediante Microsoft Visual Studio.

Resultado observado. La primera compilación permitió identificar cuatro errores CS0246 relacionados con el tipo boolean. Esta evidencia permitió reconocer las incompatibilidades producidas por la plantilla de generación antes de aplicar las correcciones necesarias.

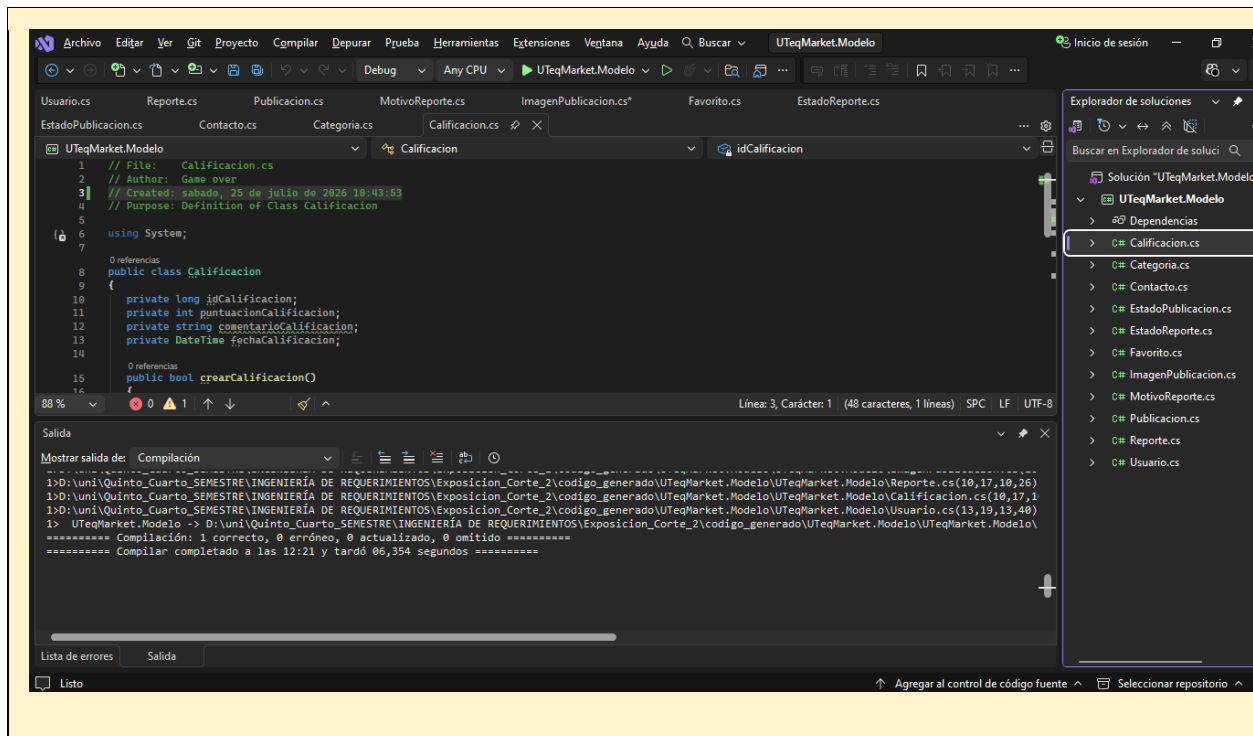


Figura 7.2. Compilación final del código C# de UTEQMarket.

Fuente: Elaboración propia mediante Microsoft Visual Studio

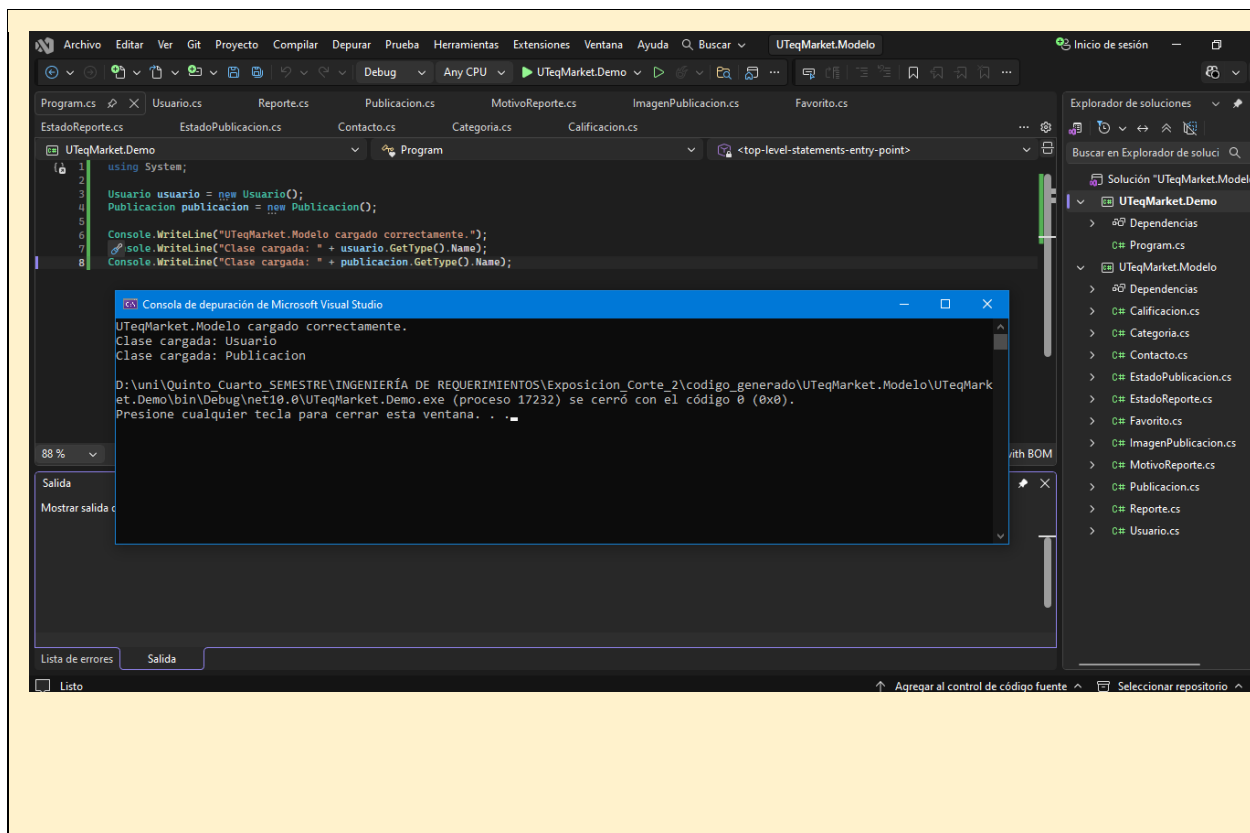
7.2 Ejecución

El proyecto UTEqMarket.Modelo es una biblioteca de clases, por lo que no posee un punto de entrada ejecutable. Para comprobar el uso de las clases debe agregarse un proyecto de consola a la misma solución y establecerlo como proyecto de inicio.

```
using System;

Usuario usuario = new Usuario();
Publicacion publicacion = new Publicacion();

Console.WriteLine("UTEqMarket.Modelo cargado correctamente.");
Console.WriteLine("Clase cargada: " + usuario.GetType().Name);
Console.WriteLine("Clase cargada: " + publicacion.GetType().Name);
```



Fuente: Elaboración propia mediante Microsoft Visual Studio

7.3 Evidencias

Evidencia	Estado	Descripción
Generación de archivos	Disponible	Figura 6.1 muestra el procesamiento de las clases y la ventana Generated Files.
Archivos .cs	Disponible	Figura 6.2 muestra los ocho archivos generados.
Primera compilación	Disponible	Figura 7.1 muestra cuatro errores CS0246 causados por boolean.
Compilación final	Disponible	La Figura 7.2 muestra la compilación correcta con cero errores.
Ejecución	Disponible	La Figura 7.3 muestra la ejecución de UTEqMarket.Demo.
Ausencia de errores	Confirmado	La compilación finalizó con cero errores y la demostración terminó con código 0.

8. Round Trip

La demostración de Round-Trip Engineering se realizó mediante un cambio controlado en el modelo y la posterior regeneración del archivo correspondiente. El cambio seleccionado consistió en corregir el nombre del atributo identificador de la clase Calificacion.

8.1 Cambio realizado

En la primera generación, el atributo identificador de Calificacion apareció en el archivo Calificacion.cs con el nombre attribute_idCalificacion1. Para mejorar la trazabilidad y mantener la correspondencia con el modelo de datos, el atributo fue renombrado en PowerDesigner como idCalificacion.

```
Antes: private long attribute_idCalificacion1;  
Después: private long idCalificacion;
```

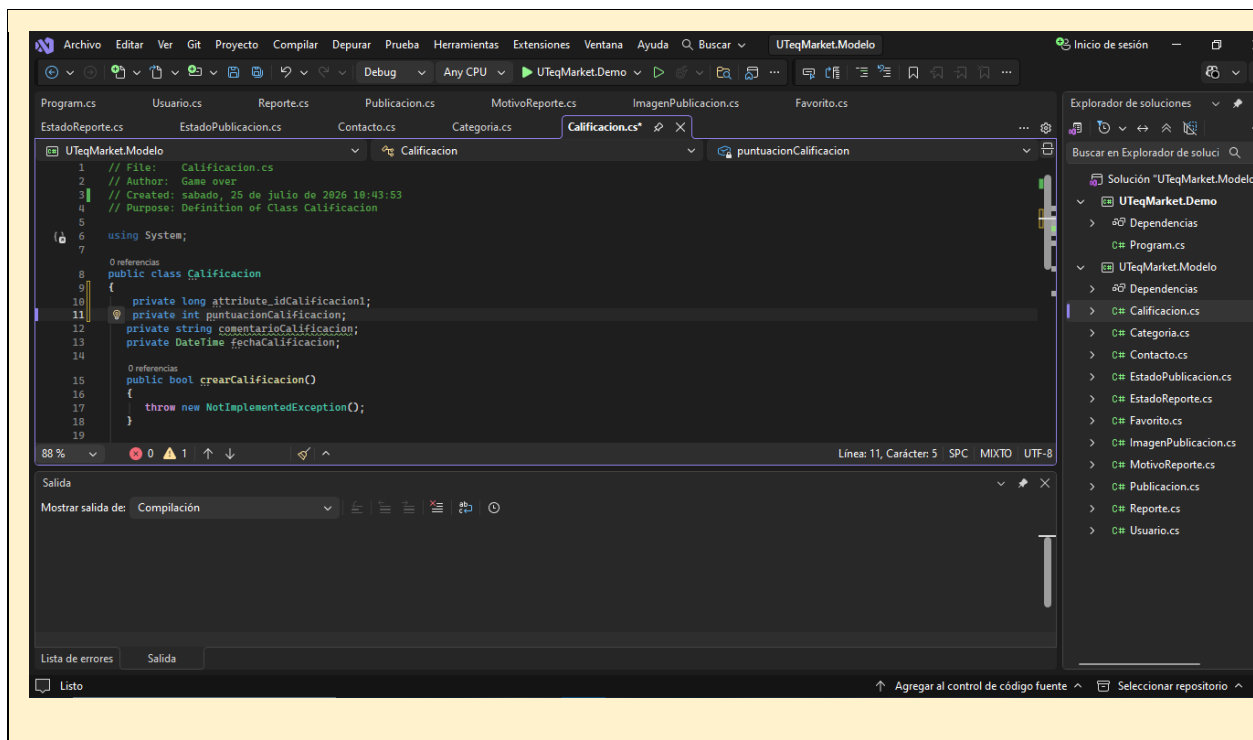


Figura 8.1. Código de Calificacion antes del cambio en el modelo.

Fuente: Elaboración propia mediante Microsoft Visual Studio

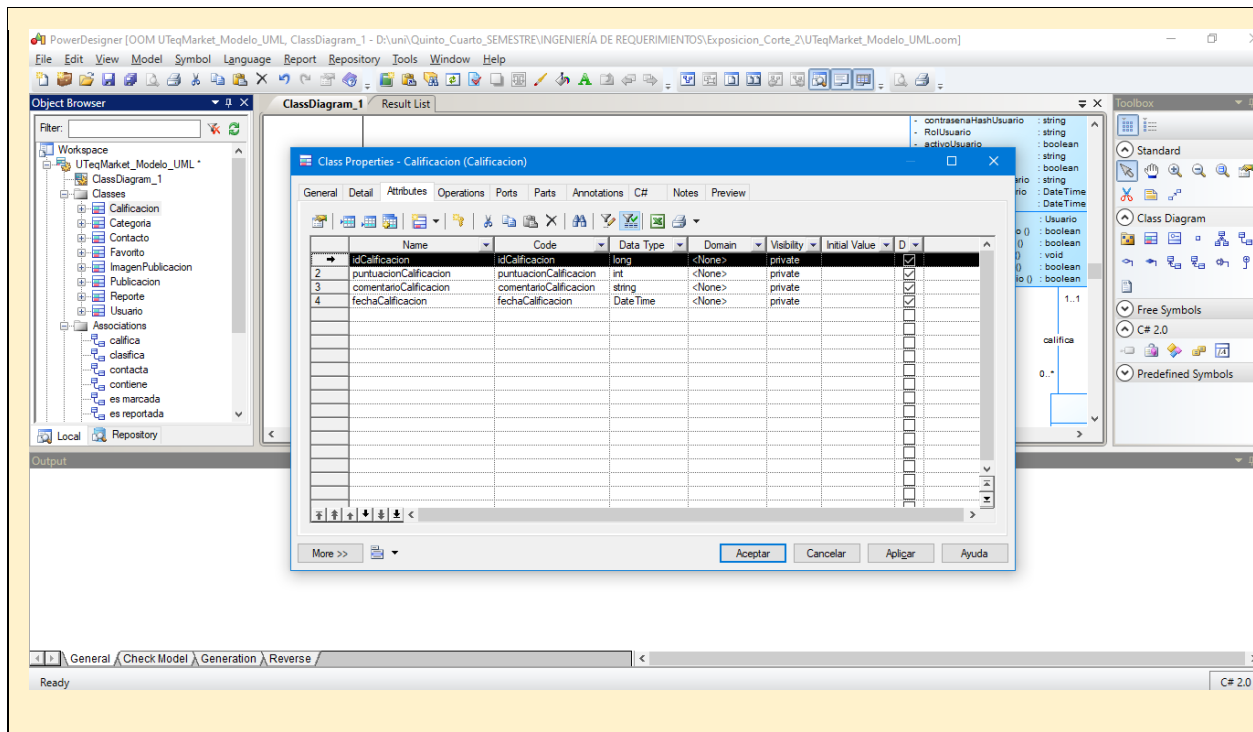


Figura 8.2. Modificación del atributo idCalificacion en PowerDesigner.

Fuente: Elaboración propia mediante PowerDesigner

8.2 Regeneración

Después de guardar y validar el modelo, se ejecutó nuevamente la generación de código C#. Para conservar la evidencia del proceso, los archivos regenerados se almacenaron en la carpeta evidencia_round_trip. La ventana Generated Files y el panel Output confirmaron la generación del archivo Calificacion.cs.

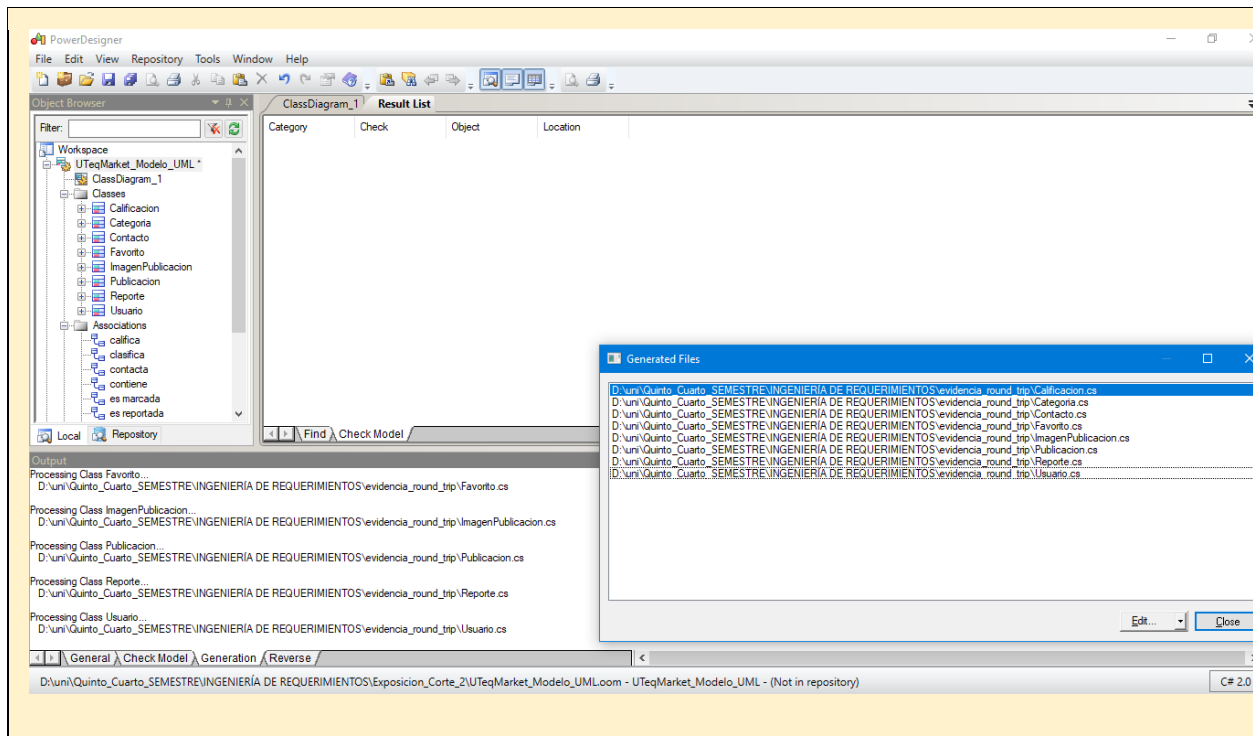


Figura 8.3. Regeneración de Calificacion.cs después del cambio en el modelo.

Fuente: Elaboración propia mediante PowerDesigner

8.3 Comparación

Elemento	Antes	Después
Nombre del campo	attribute_idCalificacion1	idCalificacion
Tipo	long	long
Visibilidad	private	private
Archivo afectado	Calificacion.cs	Calificacion.cs
Origen del cambio	Nombre automático inicial	Nombre corregido en el OOM

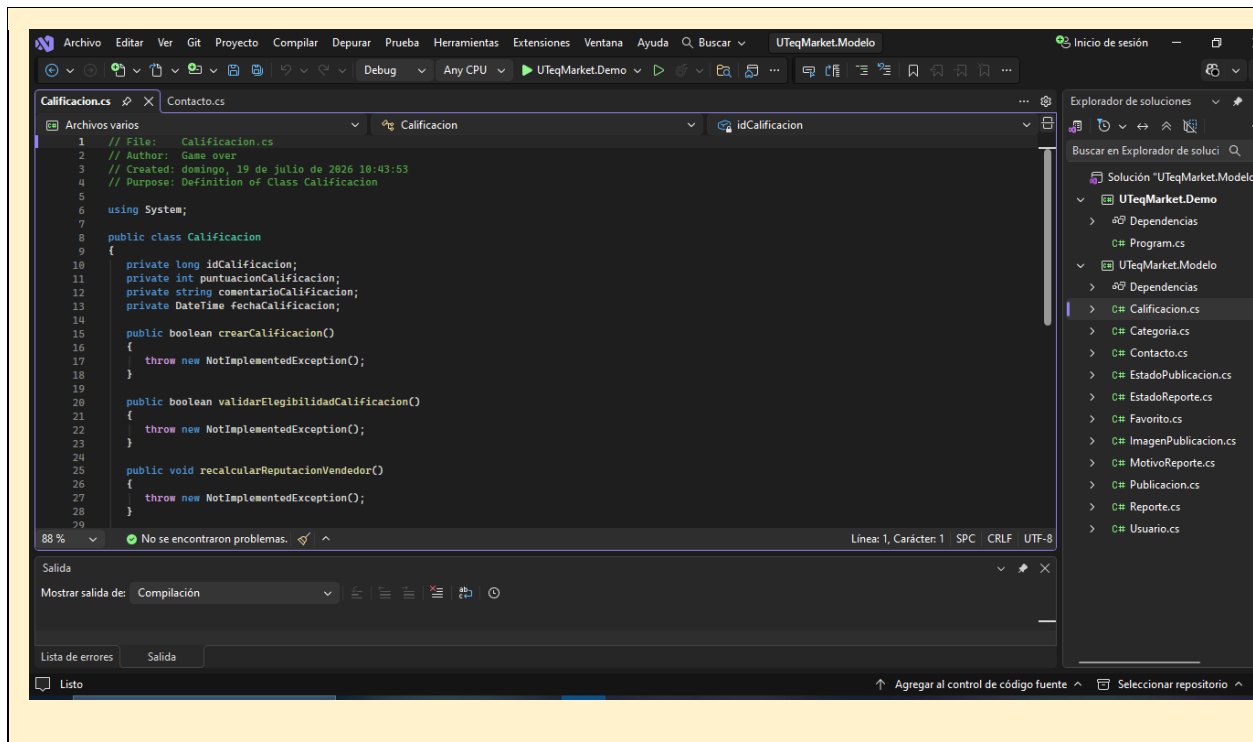


Figura 8.4. Código de Calificacion después de la regeneración.

Fuente: Elaboración propia mediante Microsoft Visual Studio

8.4 Resultado

El proceso de Round Trip se completó satisfactoriamente. El atributo generado inicialmente como `attribute_idCalificacion1` fue corregido en el Modelo Orientado a Objetos mediante la modificación de los campos Name y Code a `idCalificacion`. Posteriormente, PowerDesigner regeneró el archivo `Calificacion.cs`, en el cual se obtuvo la declaración `private long idCalificacion;`, manteniendo el tipo `long` y la visibilidad privada.

La comparación entre las Figuras 8.1 y 8.4 confirma que el cambio realizado en el modelo fue transferido correctamente al código fuente. El archivo mostrado en la Figura 8.4 conserva el tipo `boolean` producido por la plantilla de PowerDesigner; para incorporarlo al proyecto compilable de Visual Studio, este tipo debe sustituirse por `bool`.

9. Trabajo colaborativo

Esta sección debe completarse con los datos reales del Equipo D y con la evidencia del historial del repositorio. No se incorporan nombres ni commits que no consten en los archivos proporcionados.

9.1 Distribución del trabajo

Integrante	Actividades asignadas	Estado
Contreras Chávez Kevin German	Elaboración del modelo UML, generación del código C#, verificación en Visual Studio, demostración del Round Trip y documentación de la sección asignada.	Completado

9.2 Commits

Fecha	Autor	Mensaje del commit	Hash corto
Jul 20, 2026	kcontrerasc3-cloud	Add files via upload	cb947a2
Jul 20, 2026	kcontrerasc3-cloud	Delete evidencias/capturas/Kevin Contreras/Diagrama_de_clases_19-07-2026_Contreras_Kevin.jpg	2a48a44
Jul 20, 2026	avaldivezoy-hub	Fundamento teórico y justificación	436e252
Jul 18, 2026	avaldivezoy-hub	Código generado por la herramienta	7a38a92

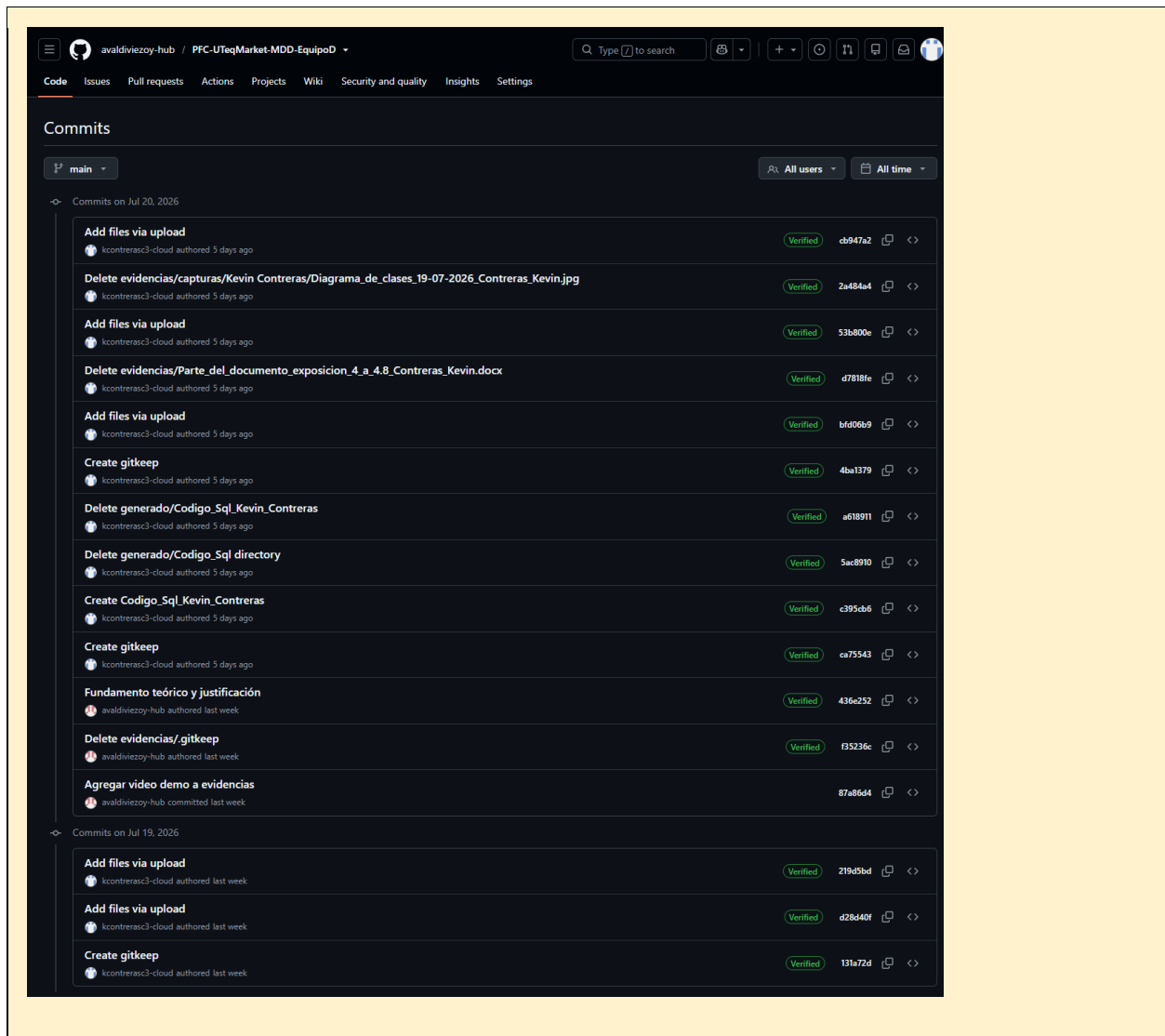


Figura 9.1. Historial de commits del repositorio de UTEqMarket.

Fuente: Elaboración propia mediante GitHub

9.3 Roles

Integrante	Rol	Responsabilidad
Contreras Chávez Kevin German	Modelador y documentador	Elaborar el modelo UML, generar y verificar el código, realizar el Round Trip y documentar las evidencias.

10. Conclusiones

Model-Driven Development: La aplicación de MDD permitió utilizar el modelo UML como artefacto principal para obtener una estructura inicial de código. Esto fortaleció la trazabilidad

entre el diseño y los archivos generados, aunque la calidad del resultado dependió de la correcta definición de tipos, nombres y relaciones.

PowerDesigner: PowerDesigner integró el modelado orientado a objetos, la validación, la generación de C# y el modelado físico para SQL Server. La herramienta facilitó la automatización, pero también evidenció la necesidad de revisar las definiciones de lenguaje y las opciones del generador.

UML: El Diagrama de Clases permitió representar de forma organizada las entidades Usuario, Publicacion, Categoria, ImagenPublicacion, Favorito, Contacto, Calificacion y Reporte. Los atributos, operaciones, estereotipos y cardinalidades sirvieron como base verificable para la generación.

Generación automática: La generación produjo ocho archivos .cs con clases, campos y firmas de métodos. Sin embargo, no implementó la lógica de negocio y generó incompatibilidades como boolean, enumeraciones faltantes y métodos con NotImplementedException, por lo que fue necesario aplicar ajustes manuales.

Aplicación en UTEqMarket: El ejercicio permitió comprobar que el módulo de publicaciones de UTEqMarket puede evolucionar desde un modelo UML hacia código C# y una estructura física de datos. La experiencia proporciona una base reutilizable para continuar con servicios, controladores, persistencia e interfaz de usuario.

Referencias

- [1] D. Lehner *et al.*, “Model-driven engineering for digital twins: a systematic mapping study,” *Softw. Syst. Model.*, vol. 24, pp. 1339–1377, 2025, doi: 10.1007/s10270-025-01264-7.
- [2] J. Michael *et al.*, “Addressing Visual Impairments with Model-Driven Engineering: A Systematic Literature Review,” *ACM Transactions on Software Engineering and Methodology*, p., 2025, doi: 10.1145/3803865.
- [3] Y. Zouani and M. Lachgar, “Zynerator: Bridging Model-Driven Architecture and Microservices for Enhanced Software Development,” *Electronics (Basel)*, p., 2024, doi: 10.3390/electronics13122237.
- [4] N. Silega, M. Noguera, Y. Rogozov, V. Lapshin, and T. González, “Transformation From CIM to PIM: a Systematic Mapping,” *IEEE Access*, vol. 10, pp. 90857–90872, 2022, doi: 10.1109/access.2022.3201556.
- [5] S. Farshidi, S. Jansen, and S. Fortuin, “Model-driven development platform selection: four industry case studies,” *Softw. Syst. Model.*, vol. 20, pp. 1525–1551, 2021, doi: 10.1007/s10270-020-00855-w.

- [6] L. Huning and E. Pulvermueller, “Automatic Code Generation of Safety Mechanisms in Model-Driven Development,” *Electronics (Basel)*, p., 2021, doi: 10.3390/electronics10243150.
- [7] Z. Su *et al.*, “MDD: A Unified Model-Driven Design Framework for Embedded Control Software,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, pp. 3252–3265, 2022, doi: 10.1109/tcad.2021.3132564.
- [8] T. Górski and J. Bednarski, “Applying Model-Driven Engineering to Distributed Ledger Deployment,” *IEEE Access*, vol. 8, pp. 118245–118261, 2020, doi: 10.1109/access.2020.3005519.
- [9] S. Rädler, L. Berardinelli, K. Winter, A. Rahimi, and S. Rinderle-Ma, “Bridging MDE and AI: a systematic review of domain-specific languages and model-driven practices in AI software systems engineering,” *Softw. Syst. Model.*, vol. 24, pp. 445–469, 2023, doi: 10.1007/s10270-024-01211-y.
- [10] A. Silva, “Model-driven engineering: A survey supported by the unified conceptual model,” *Comput. Lang. Syst. Struct.*, vol. 43, pp. 139–155, 2015, doi: 10.1016/j.cl.2015.06.001.
- [11] L. Almonte, E. Guerra, I. Cantador, and J. Lara, “Recommender systems in model-driven engineering,” *Softw. Syst. Model.*, vol. 21, pp. 249–280, 2021, doi: 10.1007/s10270-021-00905-x.
- [12] E. Syriani, L. Luhunu, and H. Sahraoui, “Systematic mapping study of template-based code generation,” *Comput. Lang. Syst. Struct.*, vol. 52, pp. 43–62, 2017, doi: 10.1016/j.cl.2017.11.003.
- [13] G. Paolone, M. Marinelli, R. Paesani, and P. Felice, “Automatic Code Generation of MVC Web Applications,” *Comput.*, vol. 9, p. 56, 2020, doi: 10.3390/computers9030056.
- [14] K. Lano and H. Siala, “Using model-driven engineering to automate software language translation,” *Automated Software Engineering*, vol. 31, pp. 1–59, 2024, doi: 10.1007/s10515-024-00419-y.
- [15] P. André and M. E. A. Tebib, “Assistance in Model Driven Development: Toward an Automated Transformation Design Process,” *Complex Syst. Informatics Model. Q.*, vol. 38, pp. 54–99, 2024, doi: 10.7250/csimq.2024-38.03.